

CS4523 Advanced Algorithms

Fixed Hash vs Randomized Universal Hash Function

Final Report

Date of Submission:

21/09/2026

Team Members:

Maathavan N. 220369J

Varun P. 220659A

1. Introduction

1.1 Background on Hash Tables

Hash tables are ubiquitous data structures in modern computer science, renowned for their ability to provide average case $O(1)$ time complexity for fundamental operations such as insertion, deletion, and search. This extraordinary efficiency makes them the backbone of dictionaries, sets, database indexing, and caching mechanisms. The core principle involves a hash function that maps a universe of potential keys to a finite array of buckets. However, because the universe of keys is invariably larger than the number of buckets, collisions, where multiple keys map to the exact same bucket, are mathematically inevitable due to the Pigeonhole Principle.

1.2 Collision Resolution via Separate Chaining

When collisions occur, the hash table must employ a resolution strategy. This study focuses on **Separate Chaining**, wherein each bucket in the hash table maintains a secondary data structure, typically a linked list. When a collision occurs, the new key is simply appended to the list at that bucket. While this guarantees that every key can be stored regardless of the load factor, it introduces a critical dependency: the time complexity of operations degrades linearly with the length of the chain. If a hash function distributes keys poorly, a single bucket could accumulate a disproportionate number of elements, transforming an expected $O(1)$ lookup into an $O(N)$ linear traversal.

1.3 The Threat of Algorithmic Complexity Attacks

For decades, many programming languages and systems relied on simple, deterministic hash functions (such as MurmurHash or standard modulo arithmetic) under the assumption that real world data is generally benign. However, this assumption is fundamentally flawed in adversarial environments. In 2011, researchers demonstrated that adversaries could craft specific payloads designed to force massive hash collisions in web applications. By submitting thousands of HTTP POST parameters that all hashed to the exact same bucket, attackers forced the servers' hash tables to degrade into $O(N)$ linked lists. This resulted in an $O(N^2)$ CPU consumption for parsing the request, allowing a single laptop to execute a Denial of Service (DoS) attack against enterprise servers.

1.4 Introduction to Universal Hashing

To immunize systems against such Algorithmic DoS attacks, the cryptographic and algorithmic communities turned to **Universal Hashing**, introduced by Carter and Wegman in 1979. Unlike a fixed deterministic function, a universal hash family represents a set of hash functions. At runtime, the application randomly selects one function from this family. The mathematical definition guarantees that for any two distinct keys, the probability of a collision is strictly bounded by $1/m$ (where m is the number of buckets). Because the specific

hash function is chosen dynamically and kept secret from the user, it becomes mathematically impossible for an adversary to craft a targeted payload that reliably forces collisions.

2. Mathematical Framework

2.1 Deterministic Hash Functions (Modulo Hashing)

The baseline function evaluated in this study is the standard Fixed Hash Function:

$$h(k) = k \bmod m$$

This function is exceptionally fast to compute and relies entirely on the inherent entropy of the key k . However, it is purely deterministic. If an adversary knows m , they can easily generate an infinite sequence of colliding keys by selecting $k_i = i * m$.

2.2 Carter-Wegman Universal Hashing

The Universal Hash Function evaluated is defined as:

$$h(k) = ((ak + b) \bmod p) \bmod m$$

Where:

- p is a prime number strictly larger than any possible key.
- a is a randomly chosen integer in the range $[1, p-1]$.
- b is a randomly chosen integer in the range $[0, p-1]$.
- m is the number of buckets.

2.3 The Role of Mersenne Primes

In our implementation, p is defined as the Mersenne prime $2^{61} - 1$. The choice of a Mersenne prime is highly intentional. Modulo operations involving arbitrary primes are computationally expensive because they require integer division. However, modulo operations with a Mersenne prime can be heavily optimized using bitwise shifts and logical AND operations. This allows the Universal Hash to approach the raw execution speed of a simple deterministic hash while providing robust cryptographic properties.

3. Objectives of the Study

The primary objectives of this exhaustive experimental analysis are:

1. **Evaluate Resilience:** To empirically demonstrate the absolute failure of deterministic hash functions when exposed to crafted adversarial payloads.
2. **Verify Theoretical Guarantees:** To prove that Universal Hashing successfully mitigates worst case collision scenarios, validating the Carter Wegman theorem in practice.
3. **Track Dynamic Evolution:** To monitor the exact trajectory of max chain lengths and total collisions at microscopic increments, revealing exactly how and when structural degradation occurs.
4. **Analyze Across Scales:** To observe how the asymptotic behaviors scale from tiny datasets ($N=10$) to massive datasets ($N=100,000$).

4. Experimental Methodology

4.1 System Architecture

The benchmarking suite was developed in Python, utilizing an object oriented Hash Table implementation that tracks its own internal state. To ensure accuracy, the table suppresses automatic resizing, locking the load factor to exactly 1.0 (where $m = N$). This forces the table to operate at maximum capacity, isolating the quality of the hash function as the sole variable influencing collisions.

4.2 Dataset Generation Strategies

Four highly distinct datasets were synthesized to evaluate performance across a spectrum of entropy:

- **Uniform:** Keys are generated using a cryptographically secure random number generator. This represents the ideal, high entropy scenario.
- **Sorted:** Keys are perfectly sequential integers (1, 2, 3... N). This represents highly structured, low entropy but benign data.
- **Adversarial:** Keys are generated as exact multiples of the table size. This represents a zero entropy, maliciously structured attack payload.
- **Near Duplicate:** Keys are drawn randomly from an extremely small pool (size $N/1000$). This represents massive data repetition and sparsity.

4.3 Evaluation Metrics

For every insertion step from 1 to N , the system records:

- **Max Chain Length Growth:** The length of the single longest linked list in the table.

- **Total Collisions Trajectory:** The cumulative sum of all collisions that have occurred up to that point.
- **Insertion and Search Times:** The precise time taken to perform operations, recorded in nanoseconds per key.

5. Results and Discussion: The Uniform Dataset

5.1 Theoretical Expectations

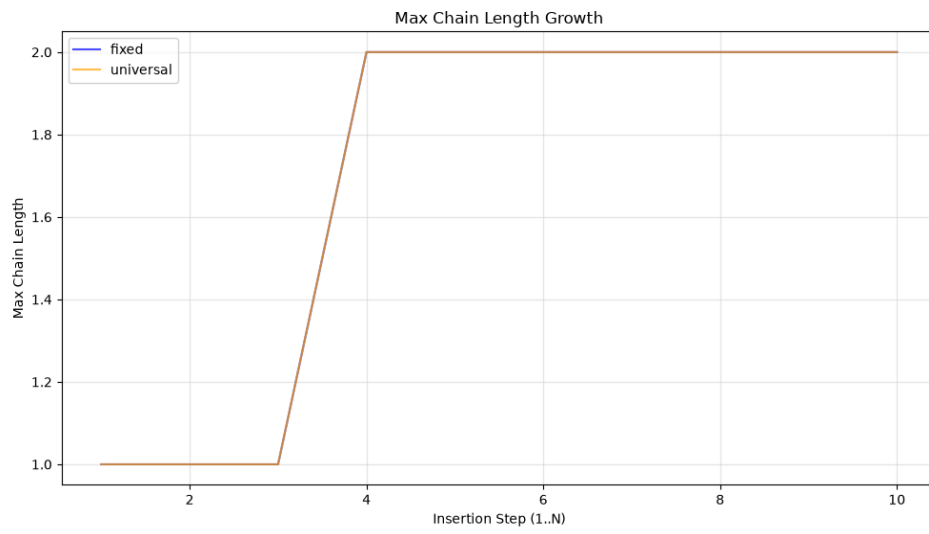
A uniformly distributed dataset provides maximal entropy. Under these conditions, the simple Fixed Hash function should act as a perfect randomizer, dispersing keys evenly across the buckets. We expect minimal collisions and a very short max chain length, closely matching the performance of the randomized Universal Hash.

5.2 Empirical Observations across Scales

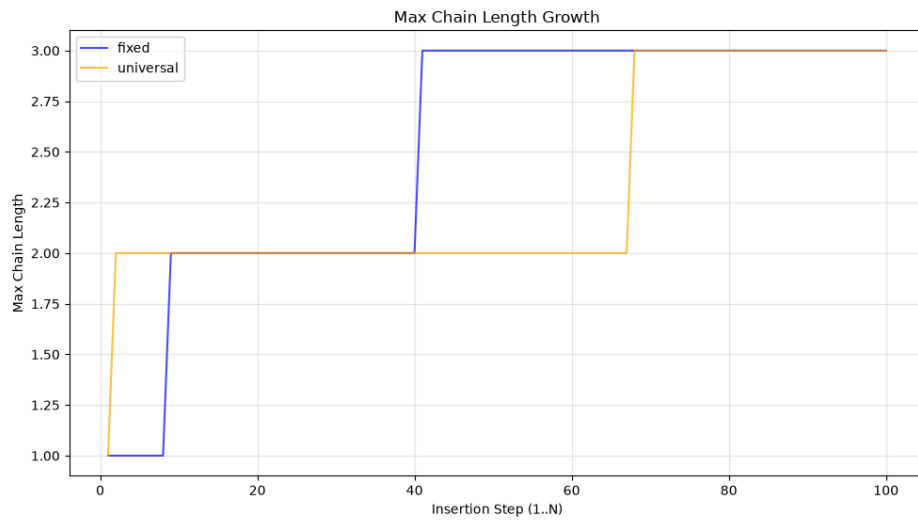
As N scales from 10 to 100,000, both the Fixed Hash and the Universal Hash maintain stable, flat trajectories. The max chain length naturally increases slightly as N grows (due to the probability of random clumps), but it strictly adheres to the expected $O(\log N / \log \log N)$ bound for random balls in bins.

Evolution of Max Chain Length ($N=10$ to 100000)

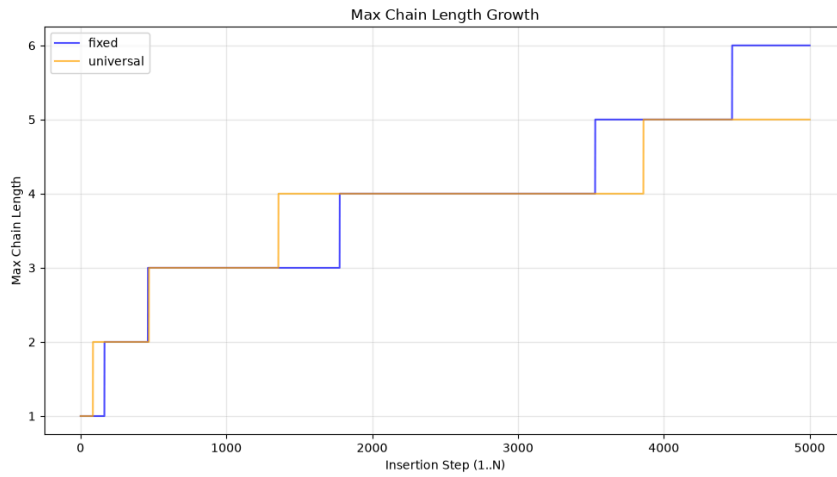
Dataset: Uniform | $n = 10$ (Max Chain Growth, $LF=1.0$)



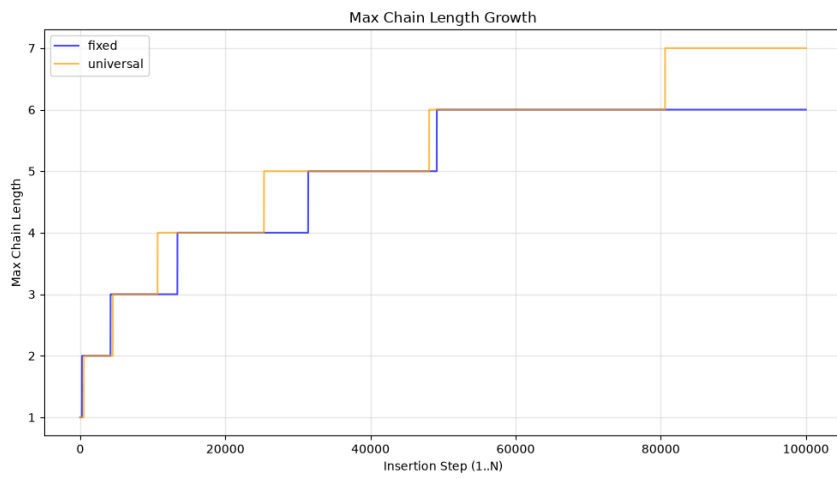
Dataset: Uniform | $n = 100$ (Max Chain Growth, $LF=1.0$)



Dataset: Uniform | $n = 5000$ (Max Chain Growth, $LF=1.0$)

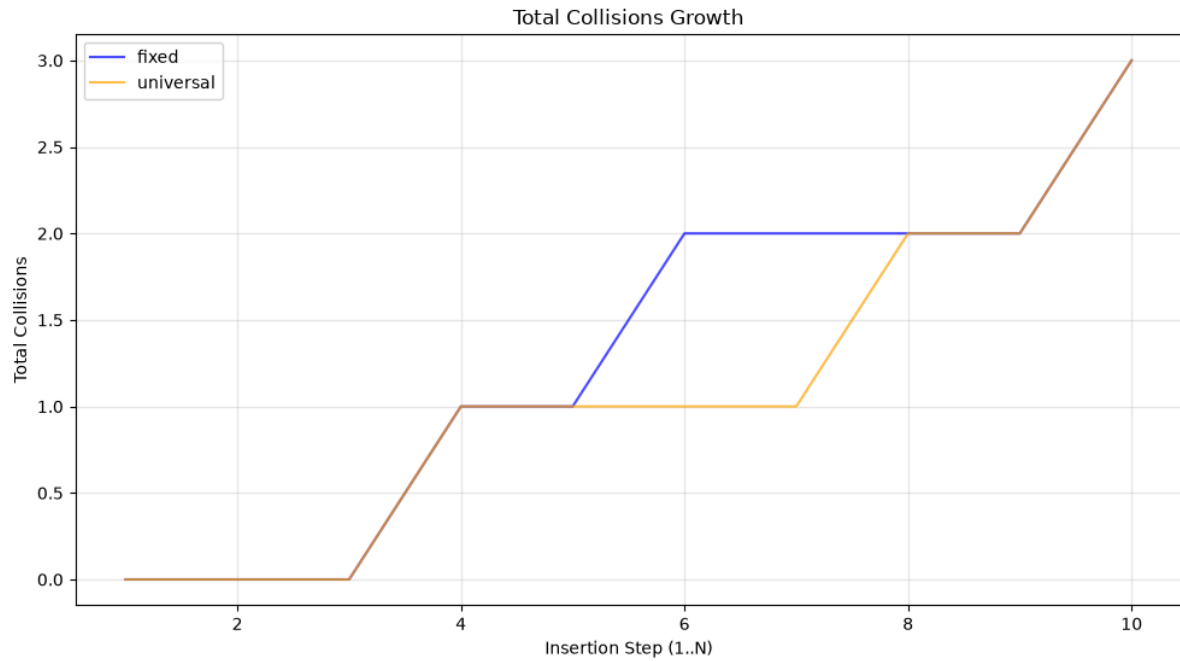


Dataset: Uniform | $n = 100000$ (Max Chain Growth, $LF=1.0$)

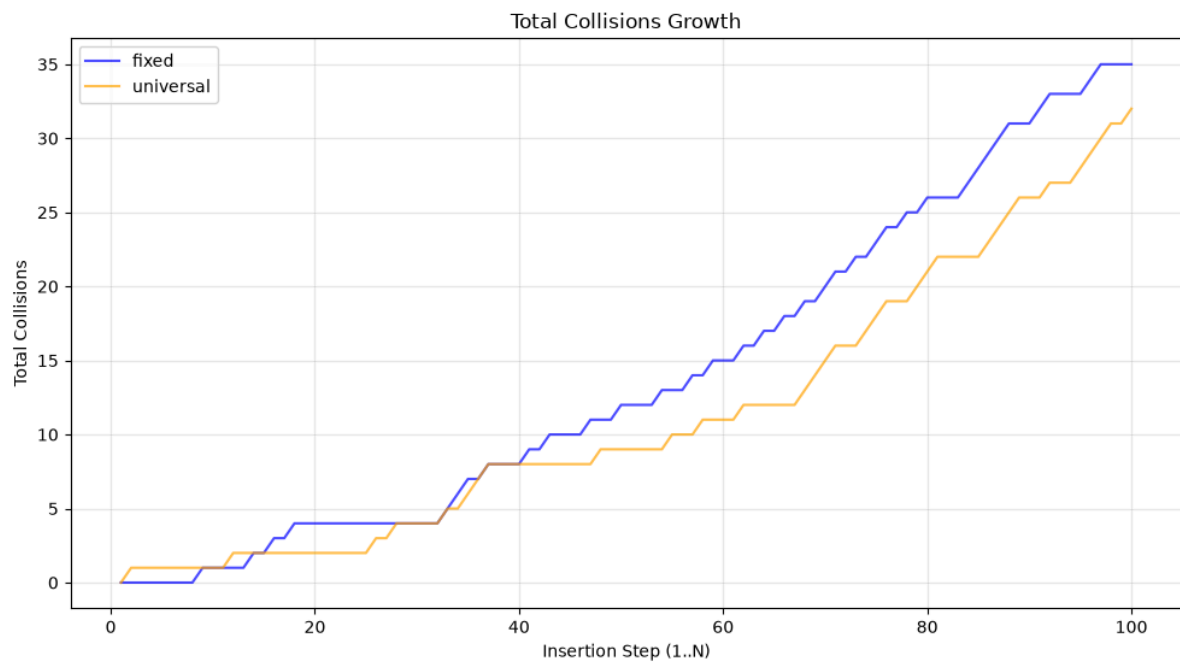


Evolution of Total Collisions (N=10 to N=100,000):

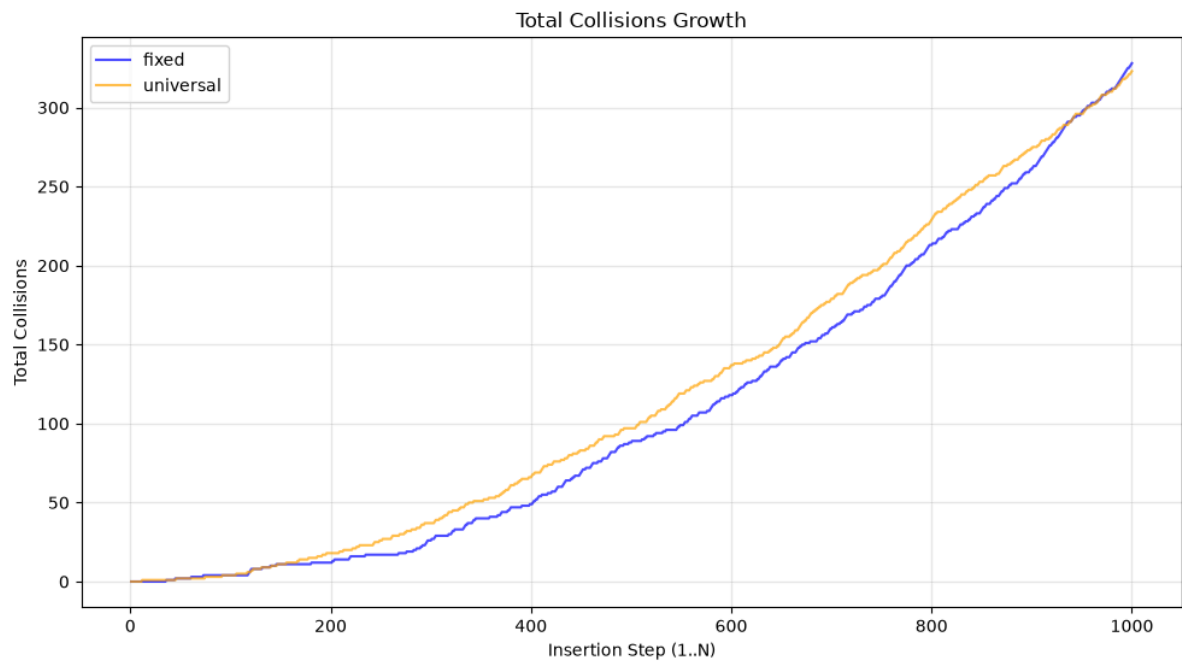
Dataset: Uniform | $n = 10$ (Total Collisions, $LF=1.0$)



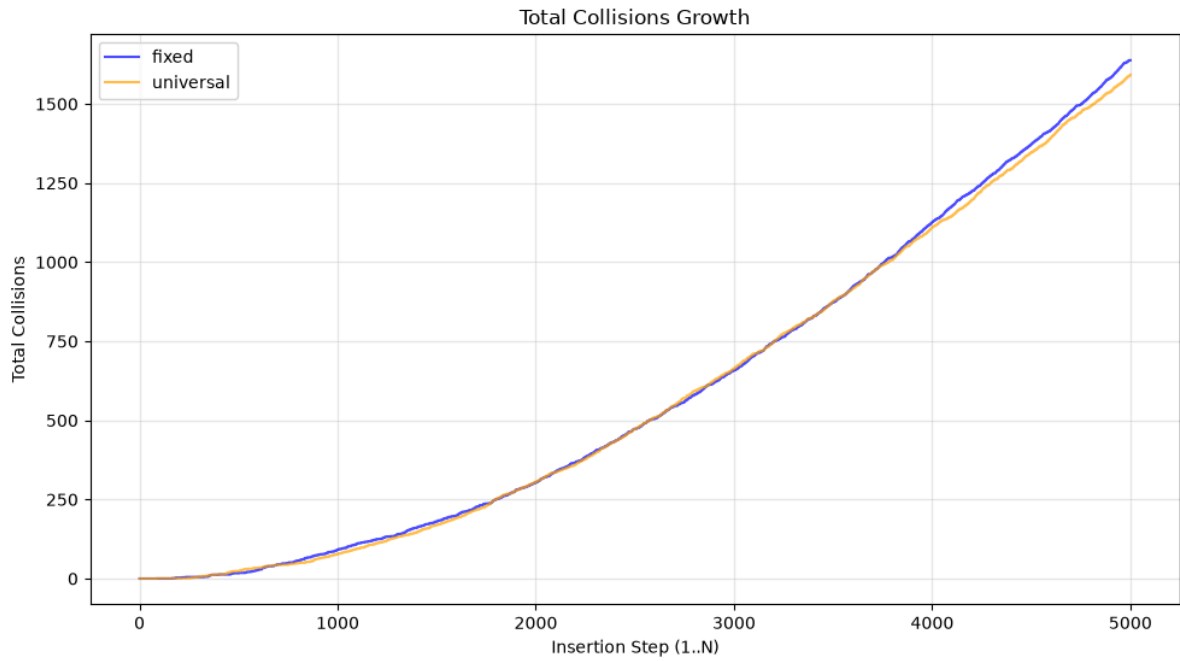
Dataset: Uniform | $n = 100$ (Total Collisions, $LF=1.0$)



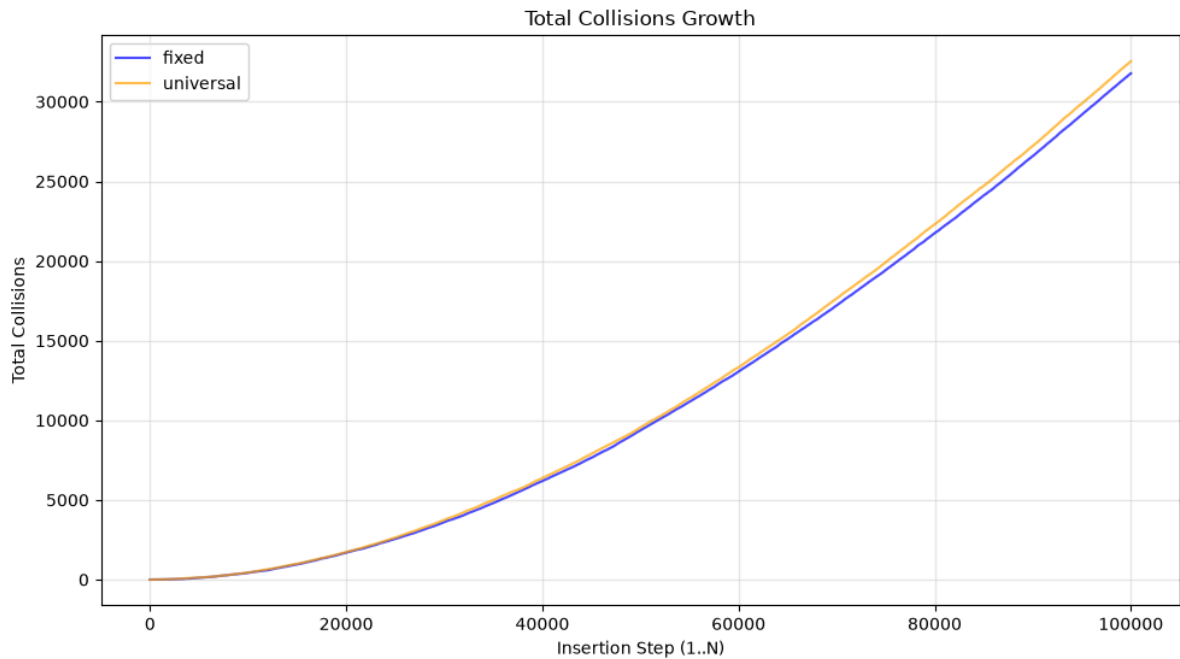
Dataset: Uniform | $n = 1000$ (Total Collisions, $LF=1.0$)



Dataset: Uniform | $n = 5000$ (Total Collisions, $LF=1.0$)

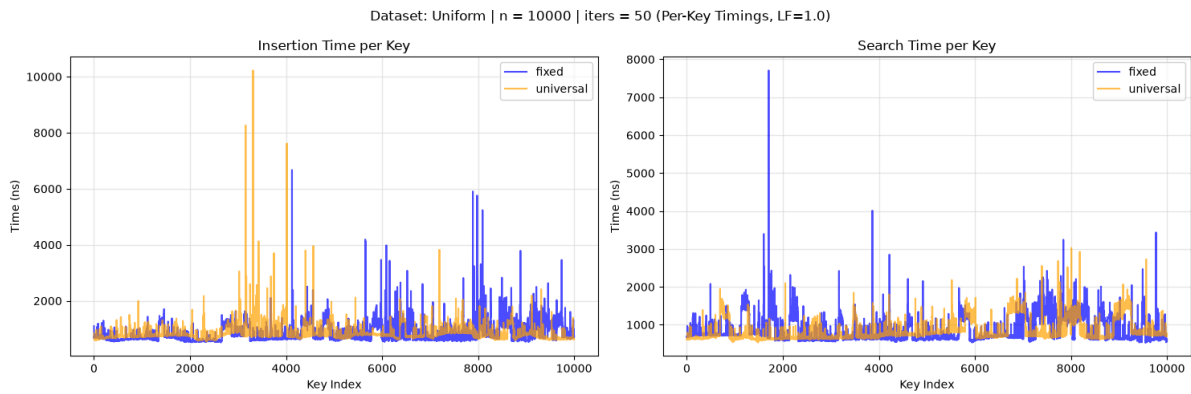


Dataset: Uniform | $n = 100000$ (Total Collisions, $LF=1.0$)



5.3 Detailed Timings Analysis

Looking at the per key execution timings (measured over 50 iterations for $N=10,000$), we see that the operational speed remains strictly constant $O(1)$. The minor vertical noise represents standard CPU jitter and caching effects, but there is zero upward trend.



Deduction: The Fixed Hash function is perfectly optimized for Uniform data. The additional mathematics of the Universal Hash (the modulo by a prime) provide no structural benefit here, meaning the Fixed Hash is the superior choice for guaranteed random distributions due to its raw computational simplicity.

6. Results and Discussion: The Sorted Dataset

6.1 Theoretical Expectations

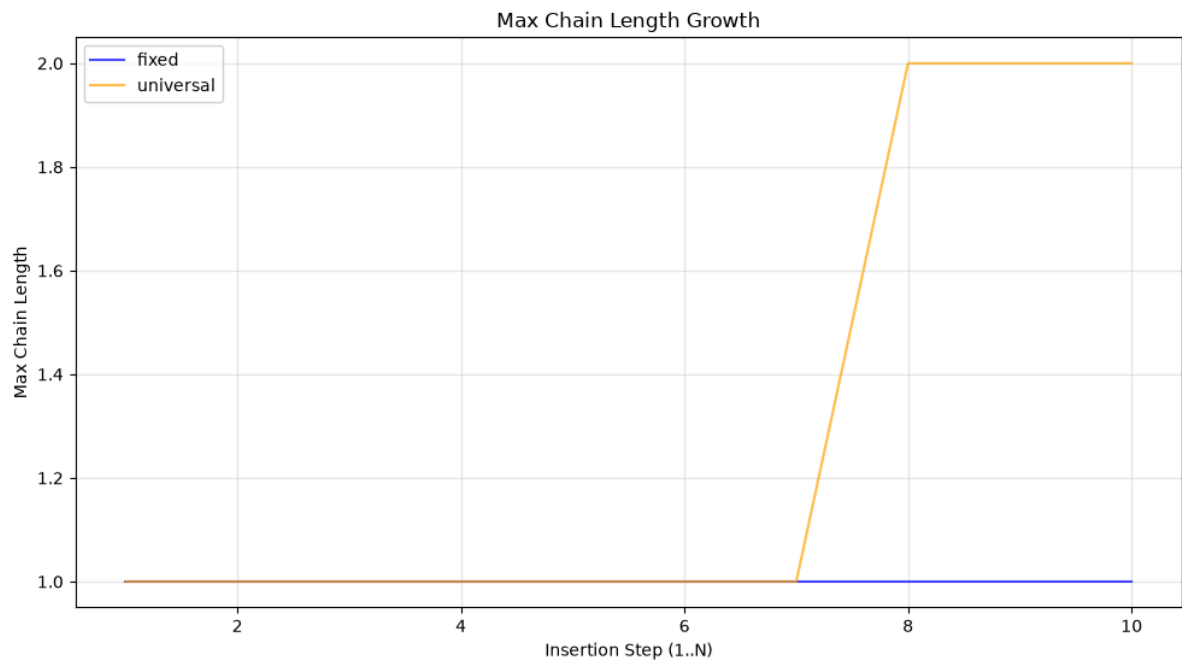
The Sorted dataset presents an interesting case. The keys are sequential (1, 2, 3...). For a modulo based Fixed Hash function where $m = N$, the mapping $k \bmod m$ perfectly distributes sequential keys into sequential, distinct buckets. This should theoretically result in exactly zero collisions.

6.2 Empirical Observations across Scales

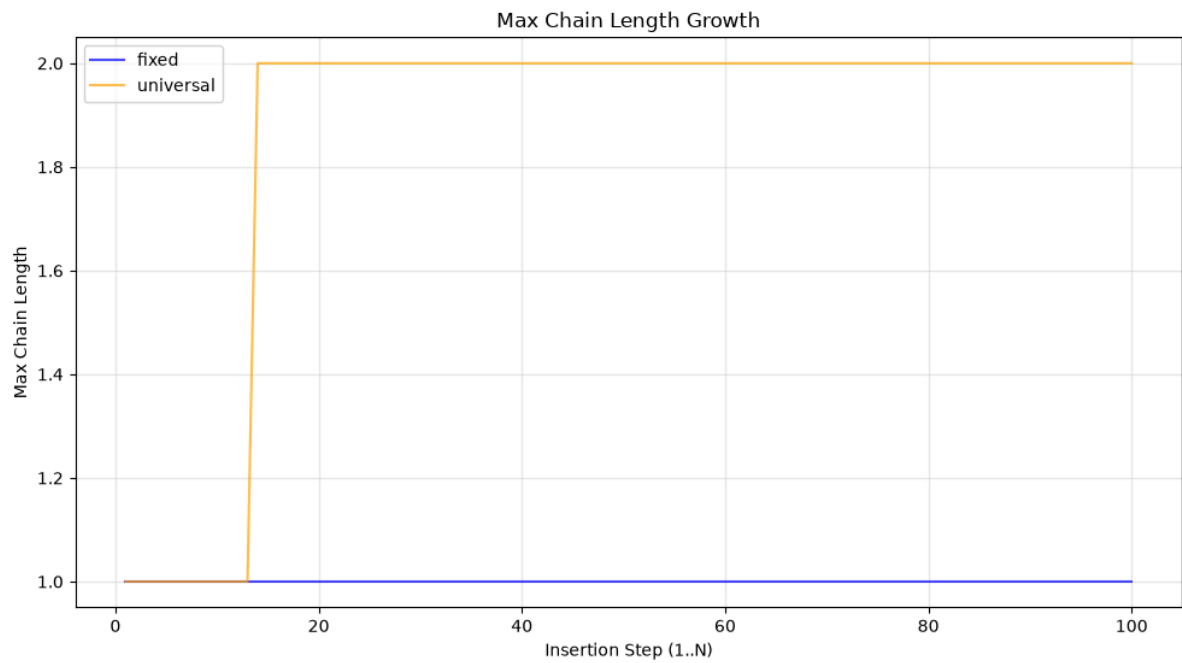
The empirical data perfectly mirrors the theory. For the Fixed Hash, the max chain length never exceeds 1, and the total collisions remain flat at 0 across all scales. Interestingly, the Universal Hash performs slightly worse in terms of structure; because it randomizes the sequential keys, it introduces standard random collisions, whereas the Fixed Hash naturally perfectly load balances them.

Evolution of Max Chain Length (N=10 to N=100,000):

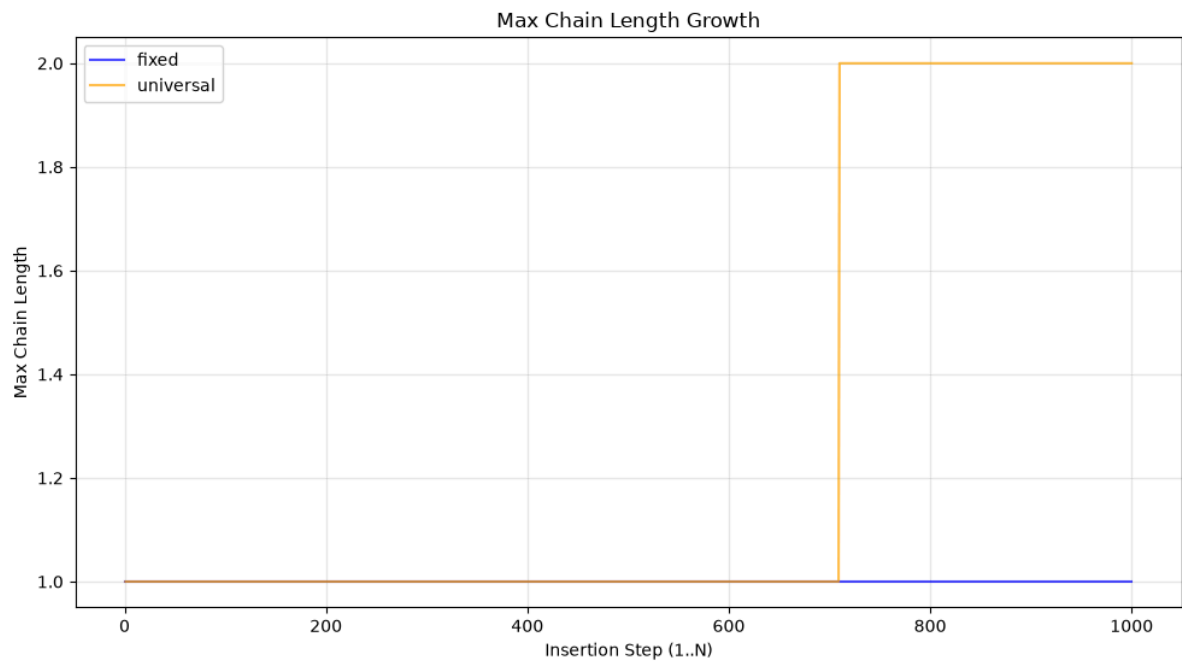
Dataset: Sorted | $n = 10$ (Max Chain Growth, $LF=1.0$)



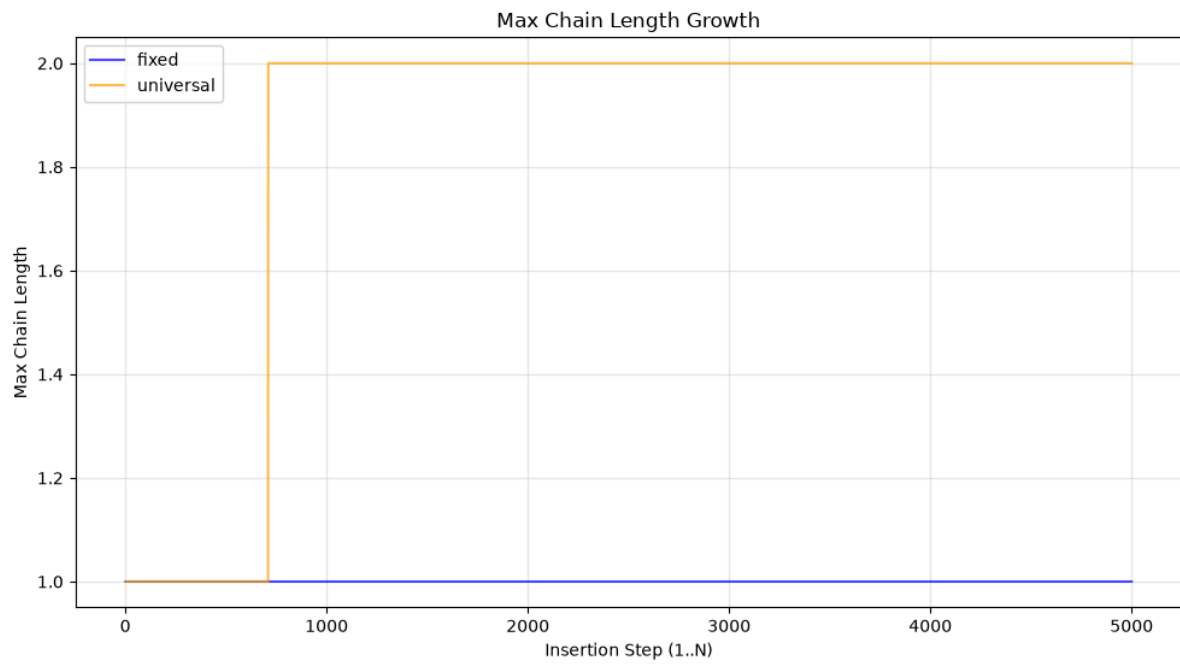
Dataset: Sorted | $n = 100$ (Max Chain Growth, $LF=1.0$)



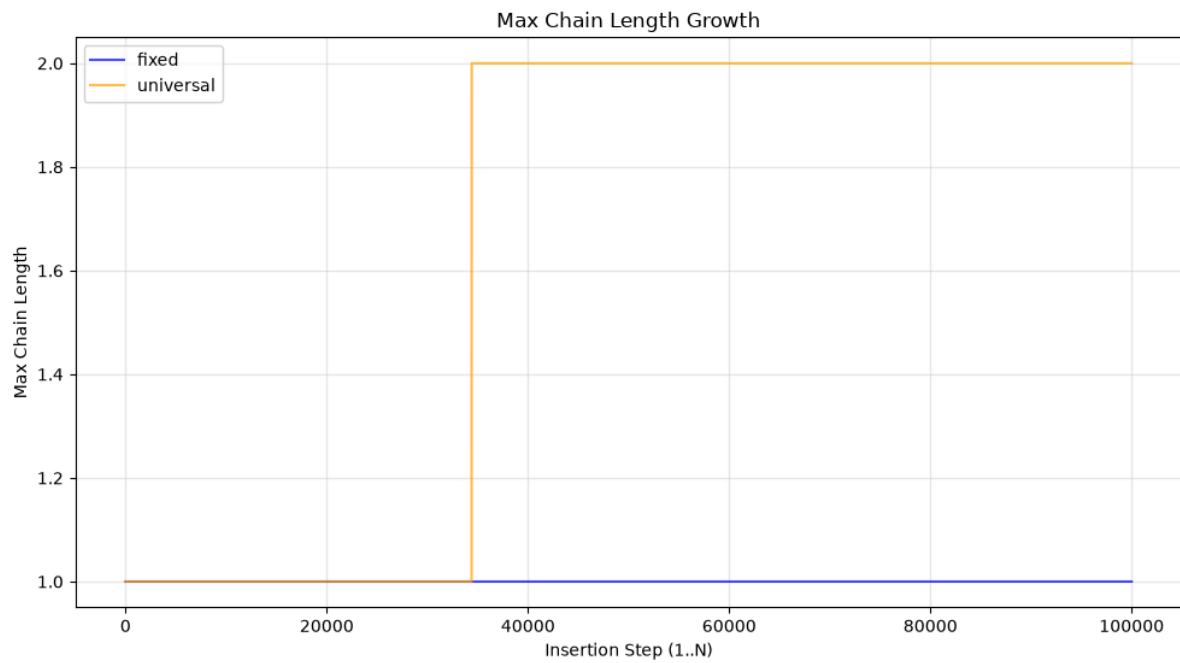
Dataset: Sorted | $n = 1000$ (Max Chain Growth, $LF=1.0$)



Dataset: Sorted | $n = 5000$ (Max Chain Growth, $LF=1.0$)

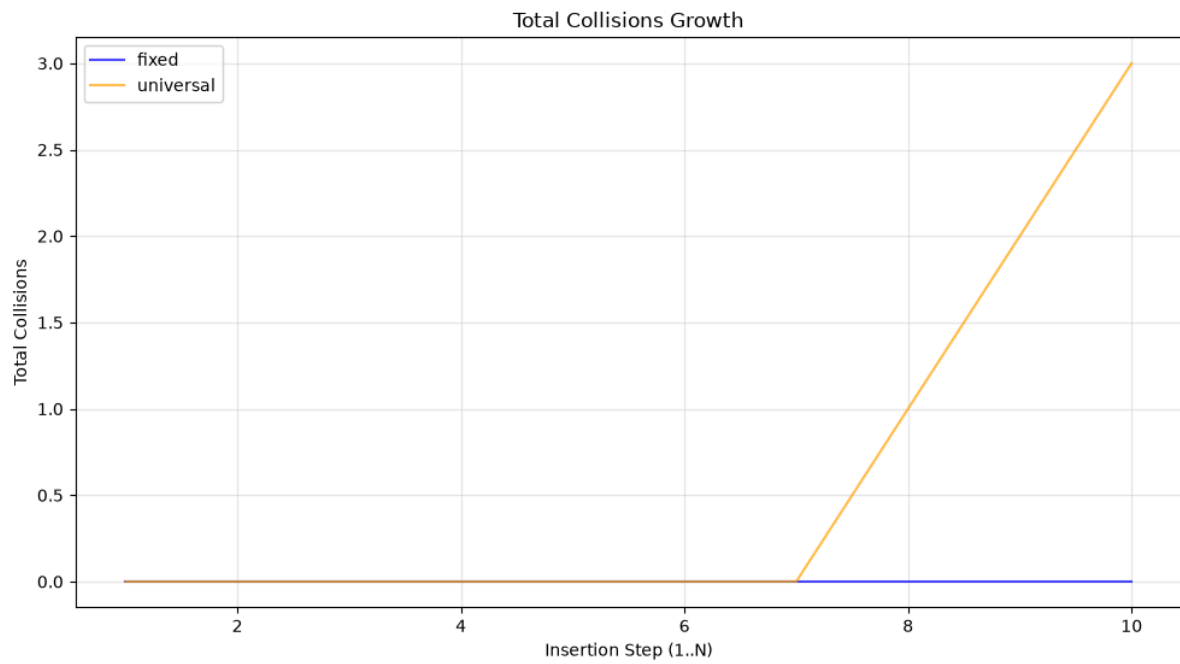


Dataset: Sorted | $n = 100000$ (Max Chain Growth, $LF=1.0$)

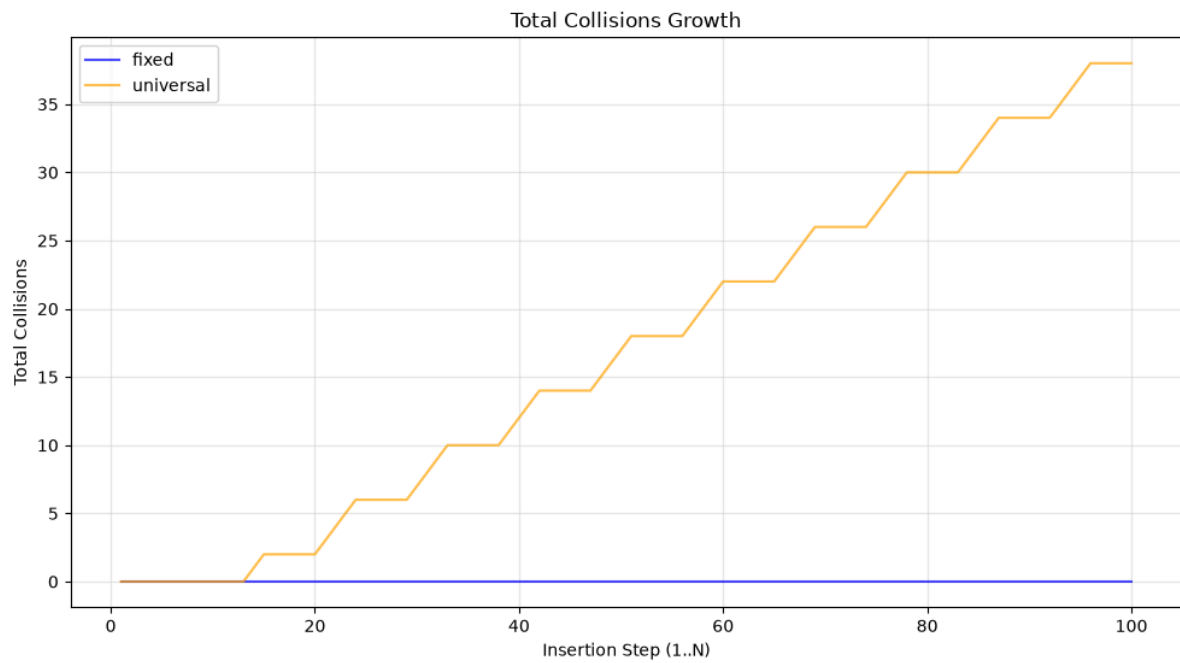


Evolution of Total Collisions (N=10 to N=100,000):

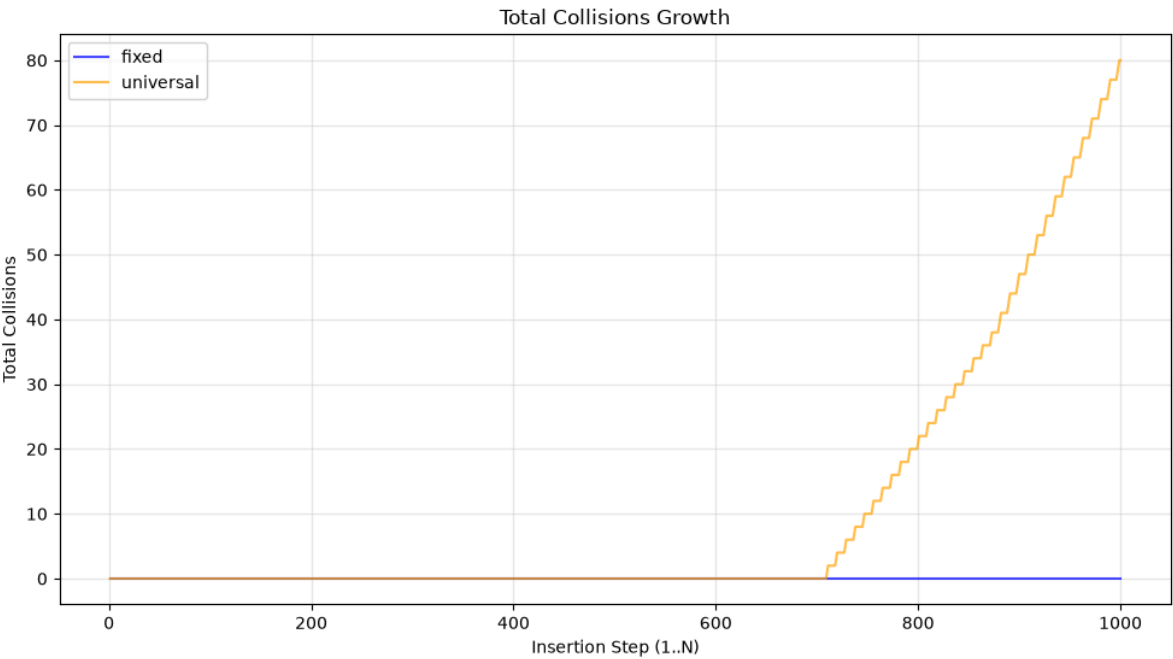
Dataset: Sorted | $n = 10$ (Total Collisions, $LF=1.0$)

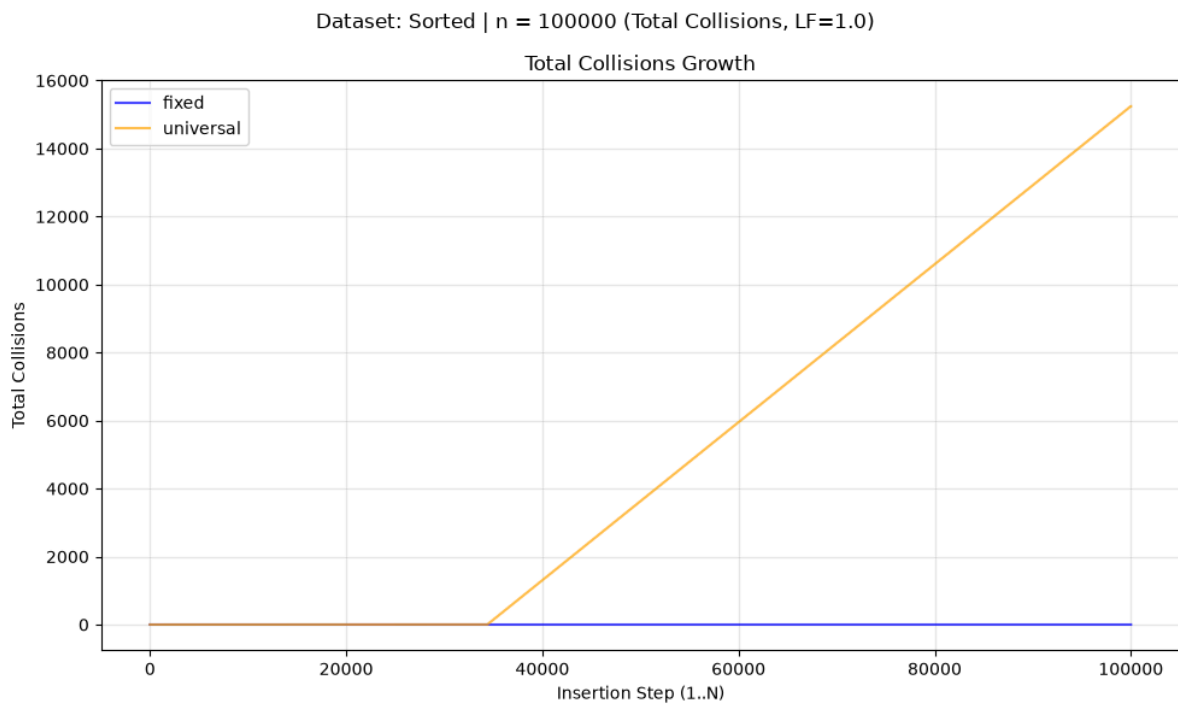
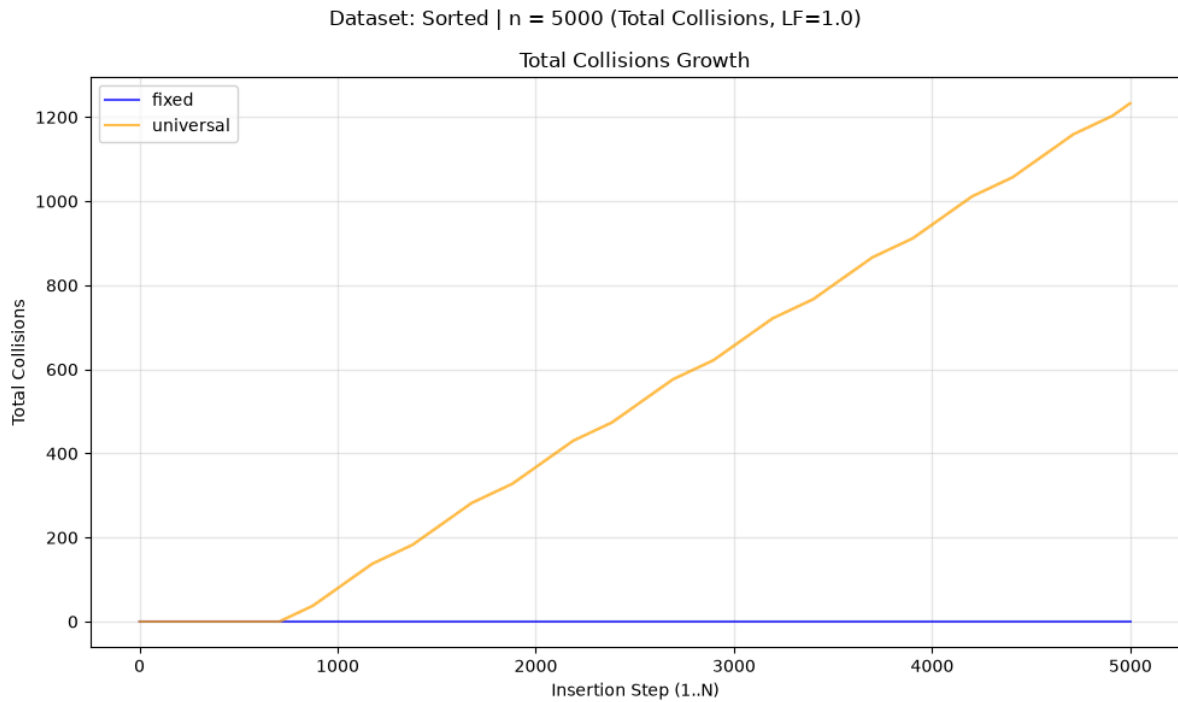


Dataset: Sorted | $n = 100$ (Total Collisions, $LF=1.0$)



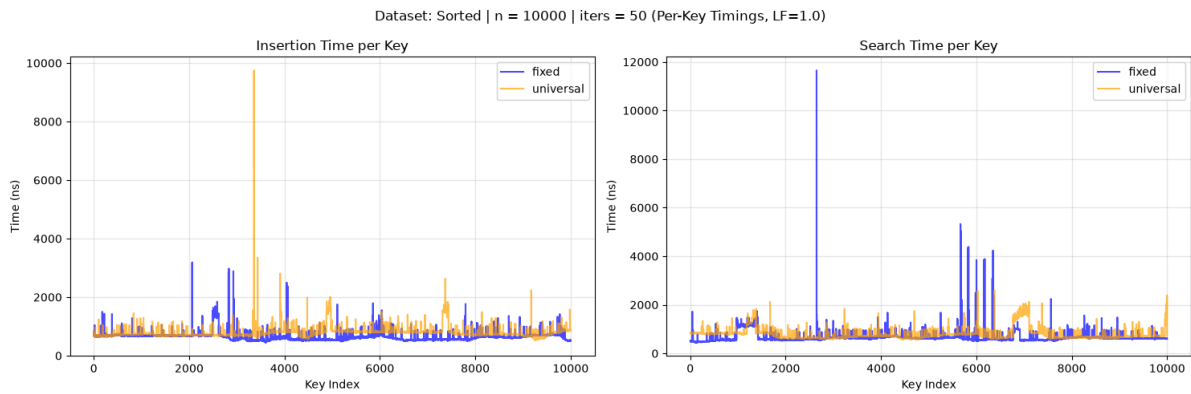
Dataset: Sorted | n = 1000 (Total Collisions, LF=1.0)





6.3 Detailed Timings Analysis

The execution times remain completely flat for both strategies. The Fixed Hash enjoys a perfectly distributed $O(1)$ behavior without any linked list traversals.



Deduction: Simple deterministic modulo hashing is incredibly synergistic with dense, sequential data. It inadvertently achieves perfect hashing for sequential integer keys, outperforming Universal Hashing's structural distribution by avoiding the collisions that randomness inherently introduces.

7. Results and Discussion: The Adversarial Dataset

7.1 The Anatomy of the Attack

The Adversarial dataset is designed to ruthlessly exploit the Fixed Hash function. Since the table size m is known (or easily guessable), the adversary sends keys that are all multiples of m . Because $(i * m) \bmod m = 0$, every single key attempts to enter bucket 0.

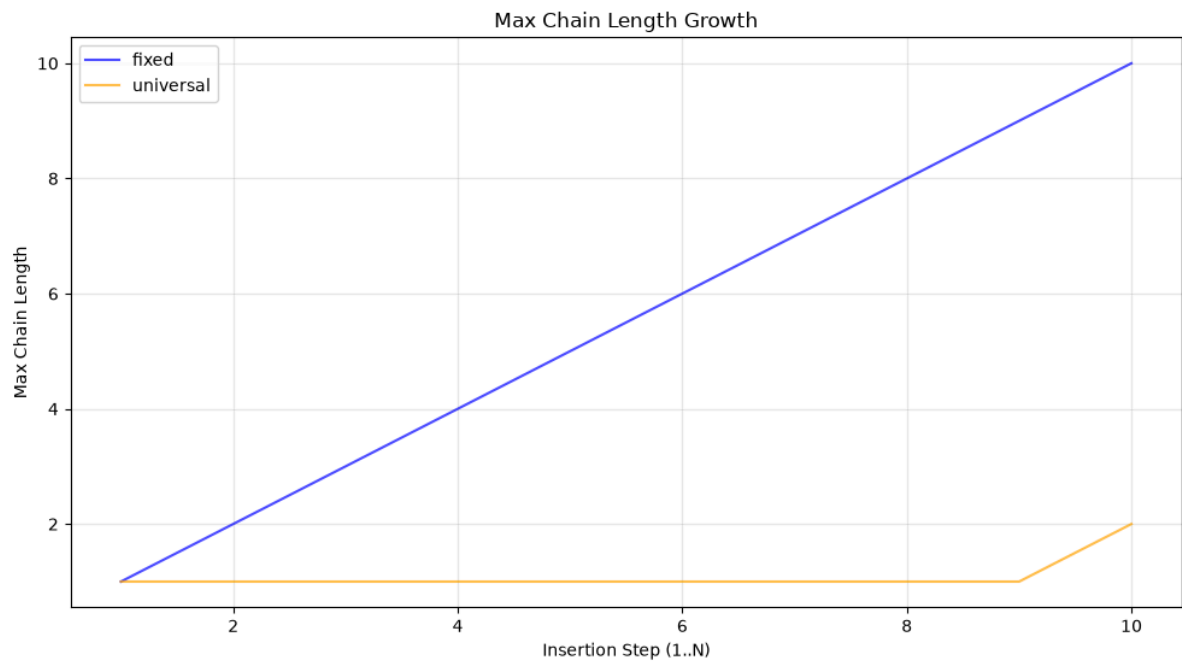
7.2 Empirical Observations across Scales

The catastrophic failure is immediate and undeniable. For the Fixed Hash (blue line), the max chain length graph is a perfect 45 degree linear slope; after N insertions, bucket 0 has a length of N . Because the hash table must traverse the chain to check for duplicate keys upon insertion, the total collision graph forms a perfect quadratic parabola.

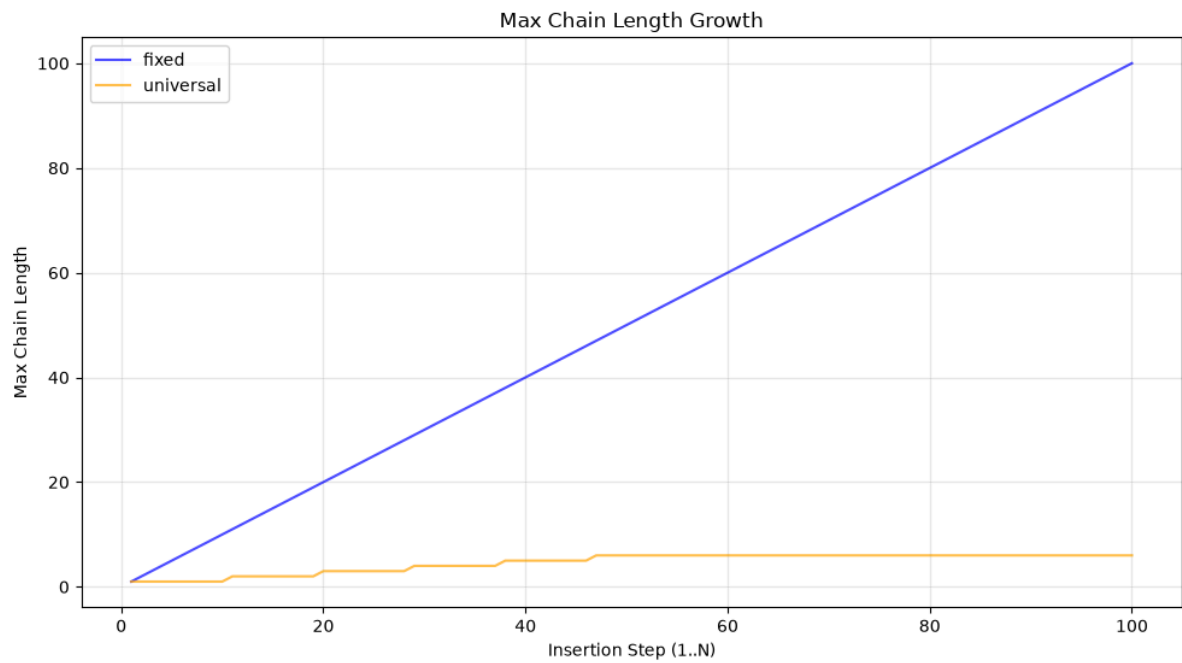
Conversely, the Universal Hash completely ignores the crafted structure. The randomized a and b parameters scramble the multiples of m , mapping them to completely random buckets. Its performance is identical to how it handled the Uniform dataset.

Evolution of Max Chain Length (N=10 to N=100,000):

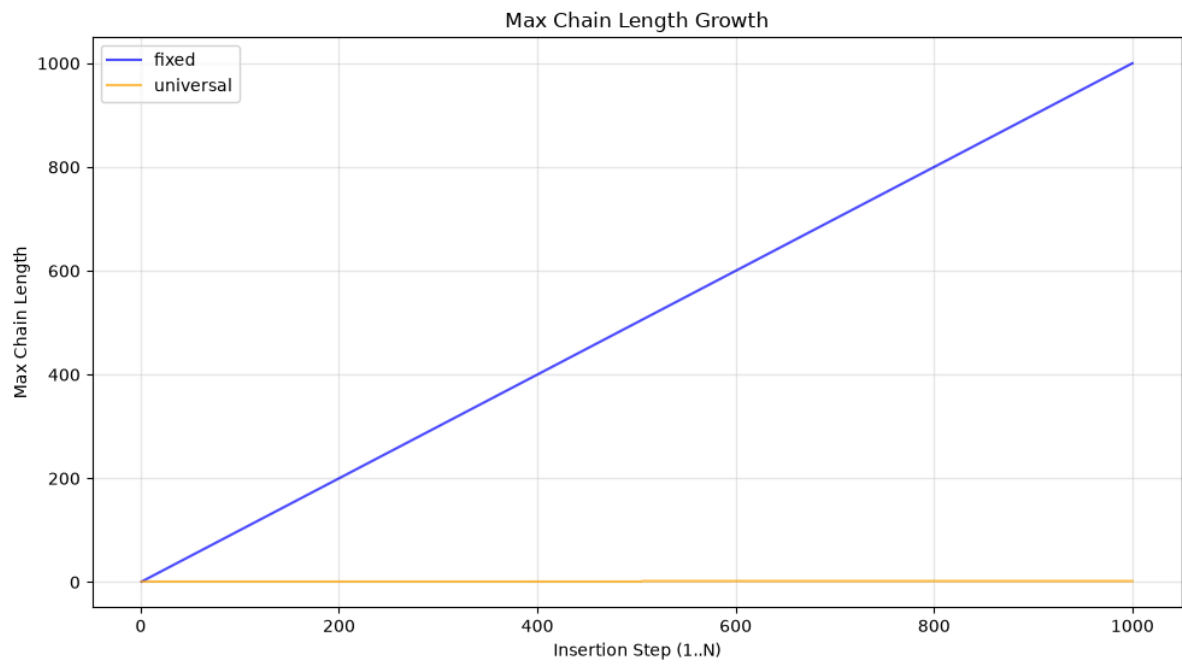
Dataset: Adversarial | $n = 10$ (Max Chain Growth, $LF=1.0$)



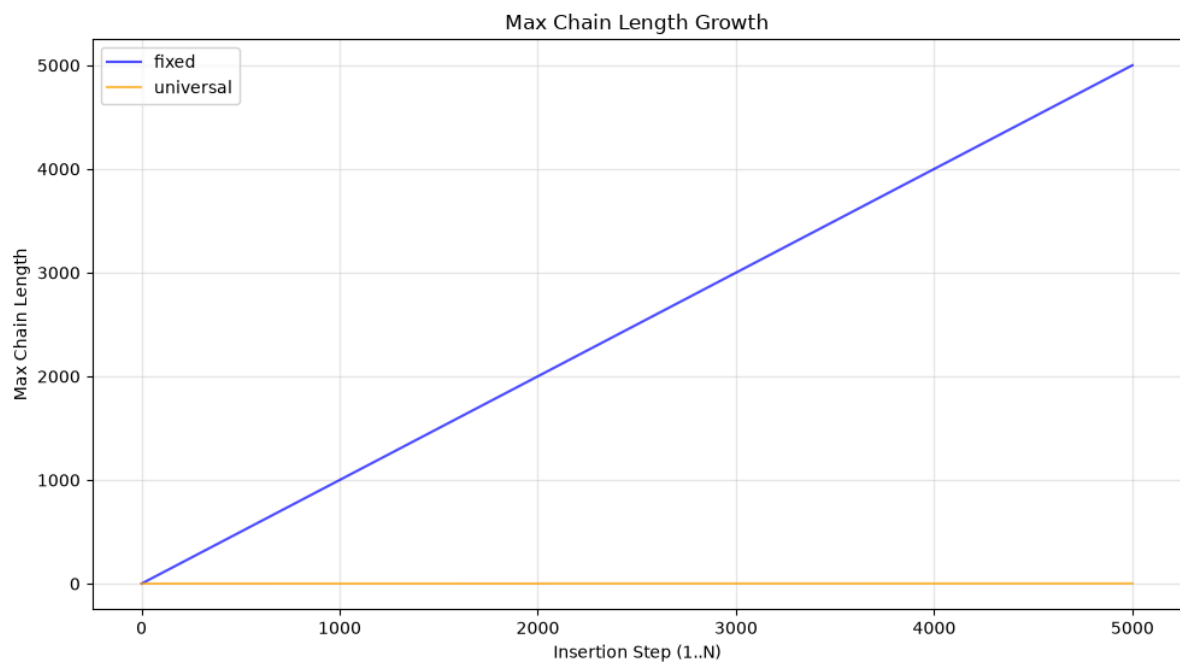
Dataset: Adversarial | $n = 100$ (Max Chain Length Growth, $LF=1.0$)



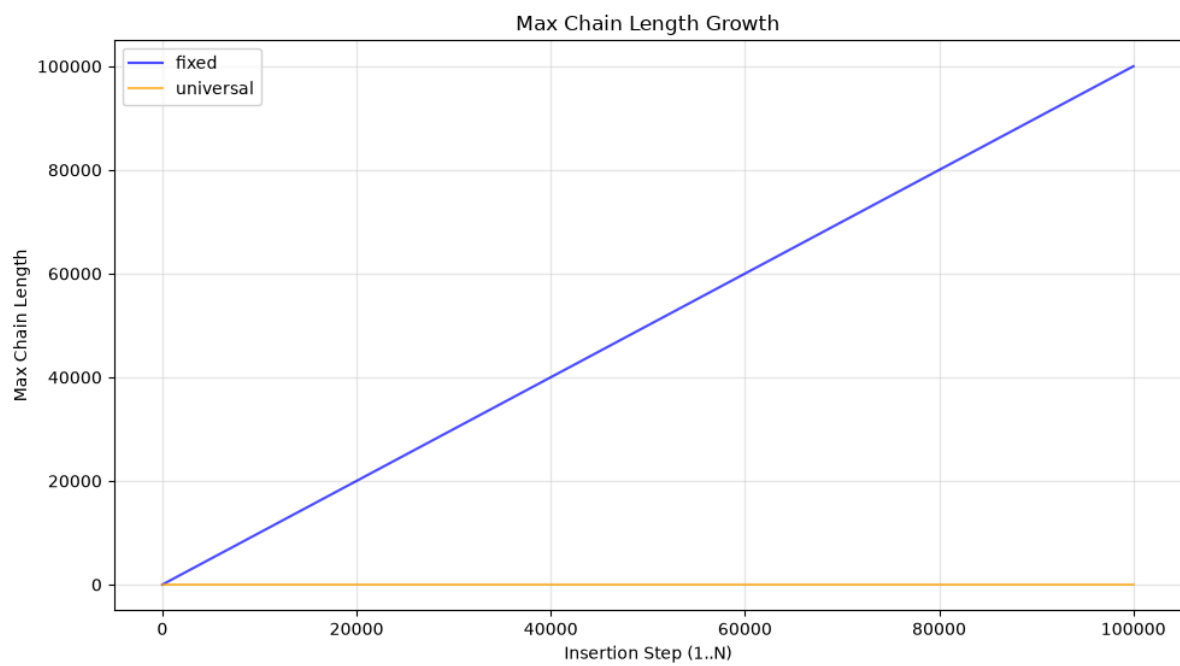
Dataset: Adversarial | $n = 1000$ (Max Chain Growth, $LF=1.0$)



Dataset: Adversarial | n = 5000 (Max Chain Growth, LF=1.0)

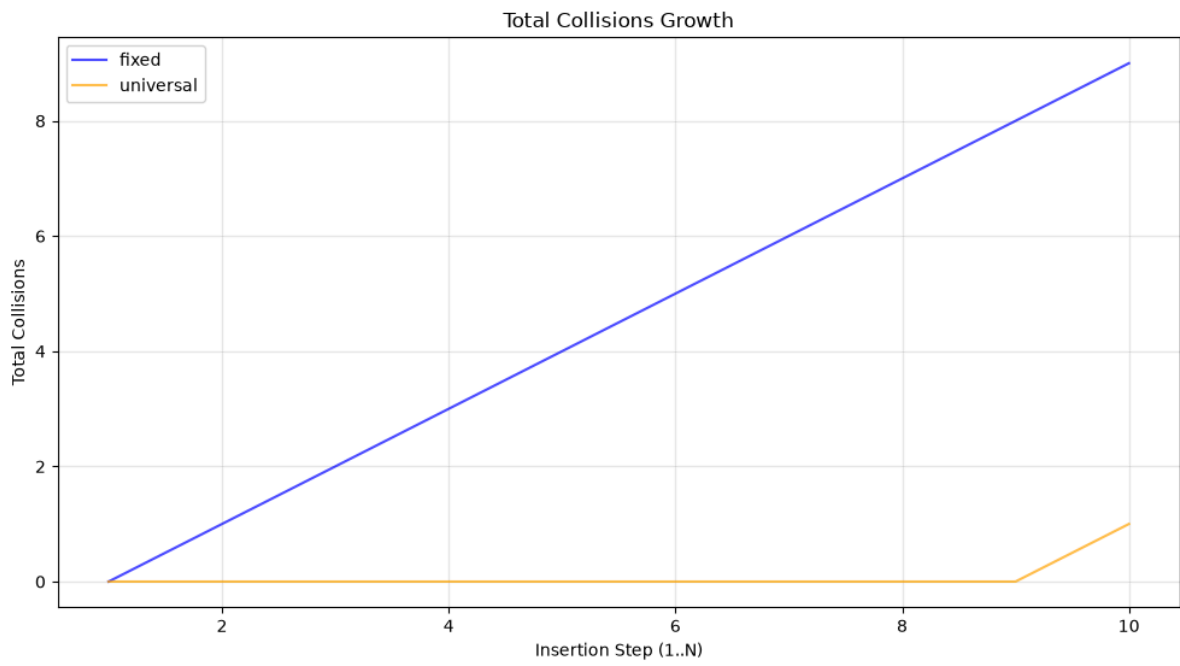


Dataset: Adversarial | n = 100000 (Max Chain Growth, LF=1.0)

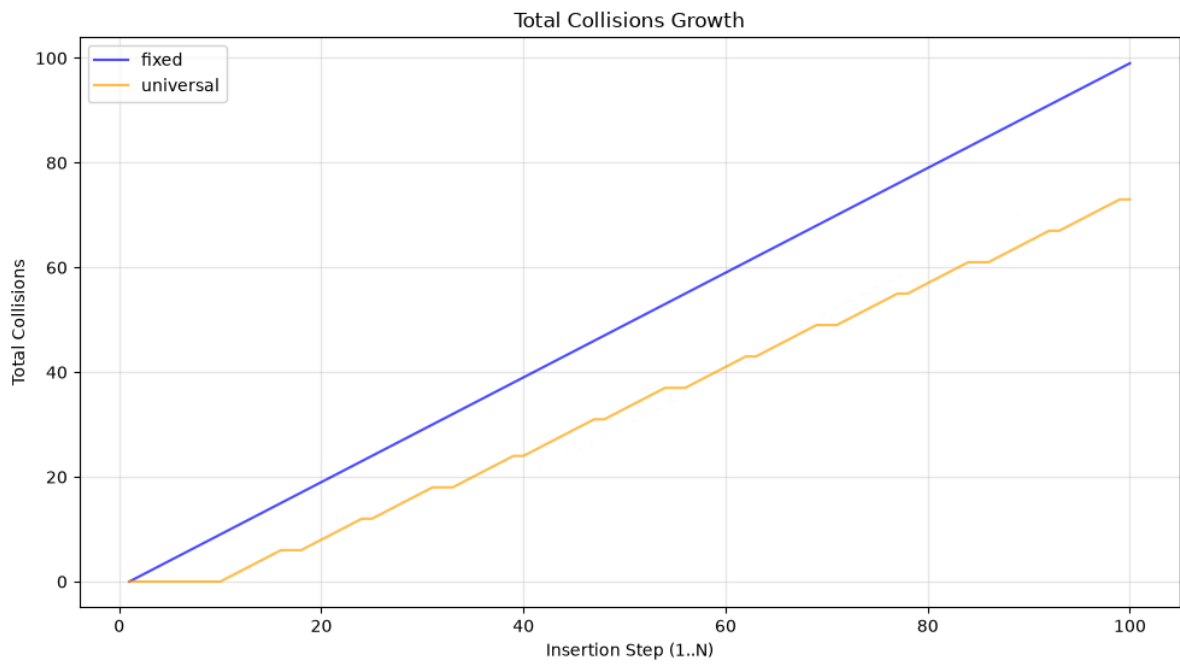


Evolution of Total Collisions (N=10 to N=100,000):

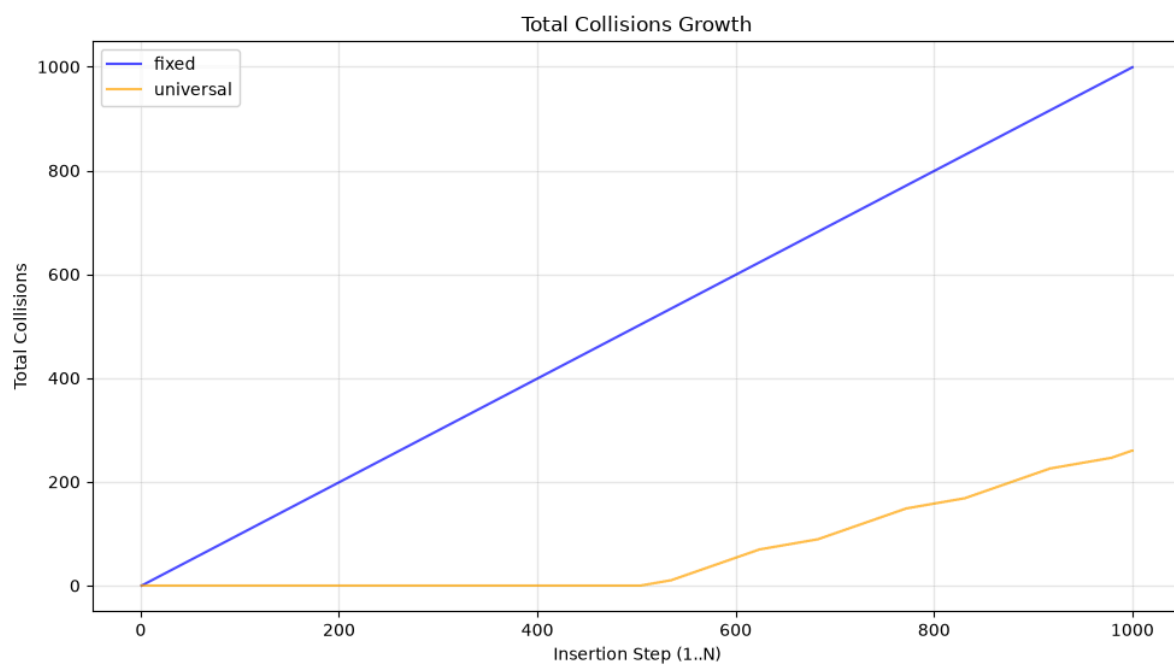
Dataset: Adversarial | $n = 10$ (Total Collisions, $LF=1.0$)



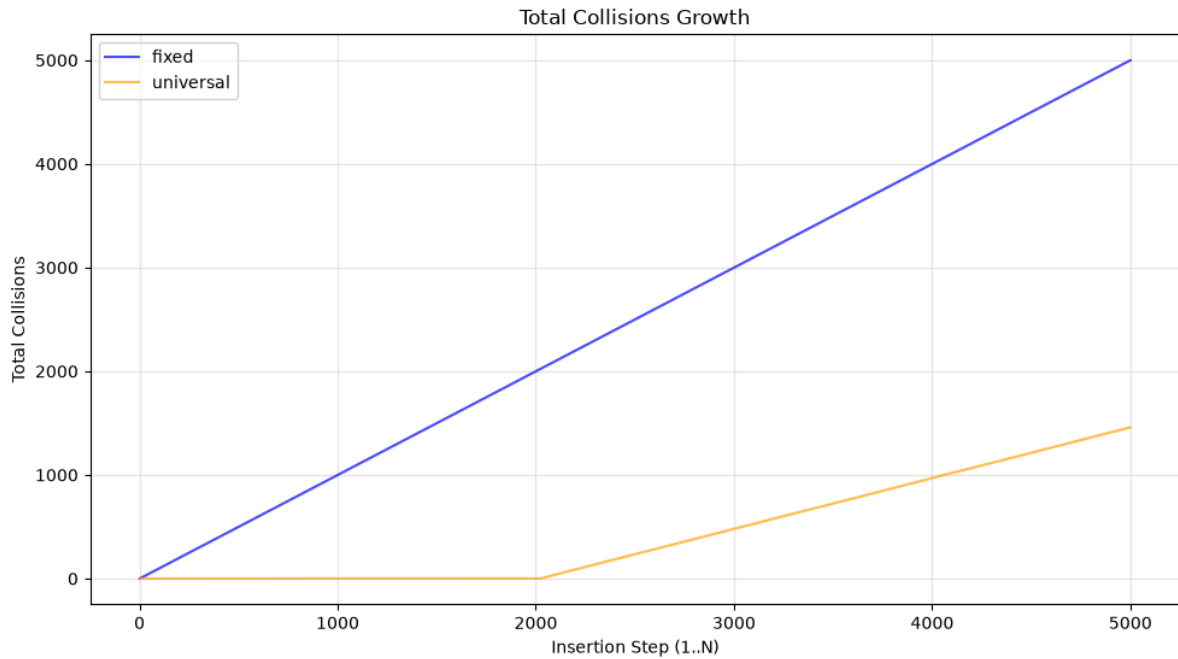
Dataset: Adversarial | $n = 100$ (Total Collisions, $LF=1.0$)



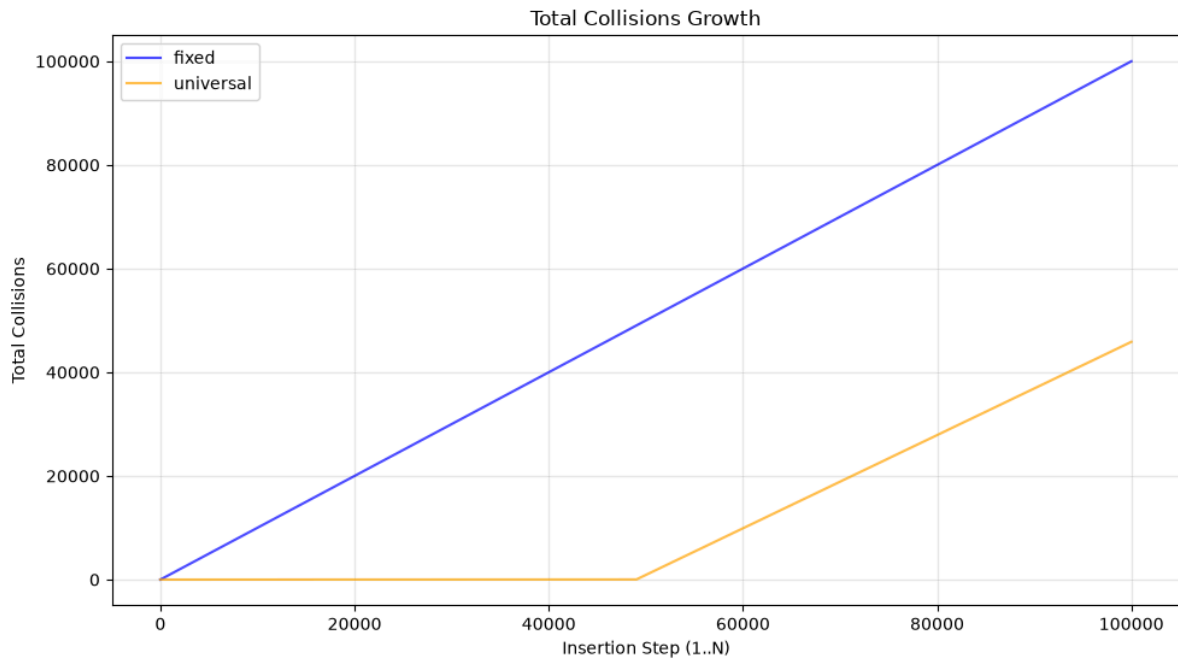
Dataset: Adversarial | $n = 1000$ (Total Collisions, $LF=1.0$)



Dataset: Adversarial | n = 5000 (Total Collisions, LF=1.0)

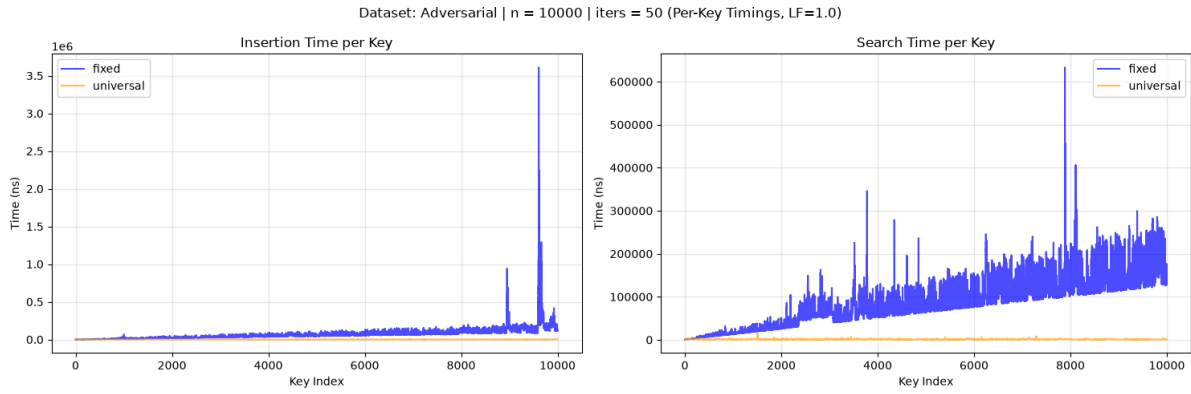


Dataset: Adversarial | n = 100000 (Total Collisions, LF=1.0)



7.3 Detailed Timings Analysis

The timings graph for the Adversarial case is the most critical visual in this report. The Fixed Hash insertion and search times degrade entirely into $O(N)$ linear time. Every subsequent key takes significantly longer to process than the last, resulting in an $O(N^2)$ total execution time. At $N=100,000$, this fundamentally halts application execution. The Universal Hash remains flat at $O(1)$.



Deduction: Deterministic hashing is a massive security vulnerability. In any environment where users can control the input data (web parameters, JSON payloads, HTTP headers), a Fixed Hash function exposes the system to trivial and devastating Denial of Service attacks.

Deduction: Universal Hashing is the definitive solution to Algorithmic DoS. It mathematically ensures that no deterministic attack payload can succeed, restoring $O(1)$ performance even under active exploitation.

8. Results and Discussion: The Near Duplicate Dataset

8.1 Theoretical Expectations

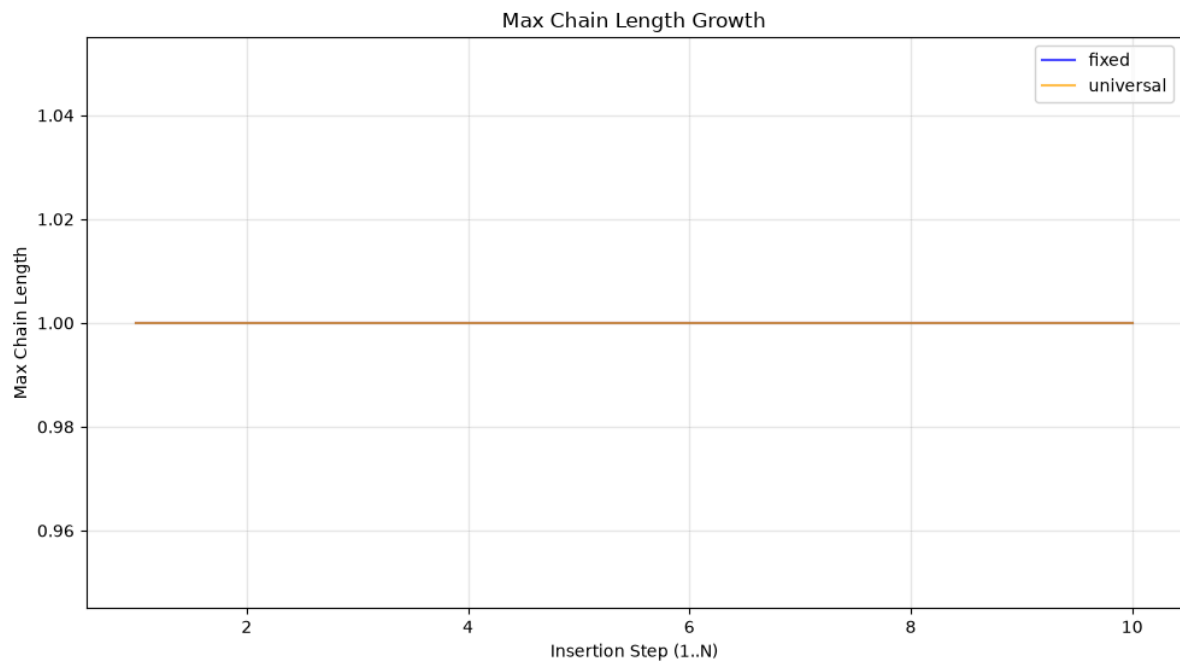
In the Near Duplicate dataset, the keys are drawn from a pool of size $N/1000$. For $N=100,000$, there are only 100 unique keys being inserted 100,000 times. Because hash tables inherently manage sets (checking for duplicates before insertion), the actual number of nodes created in the table will only be 100.

8.2 Empirical Observations across Scales

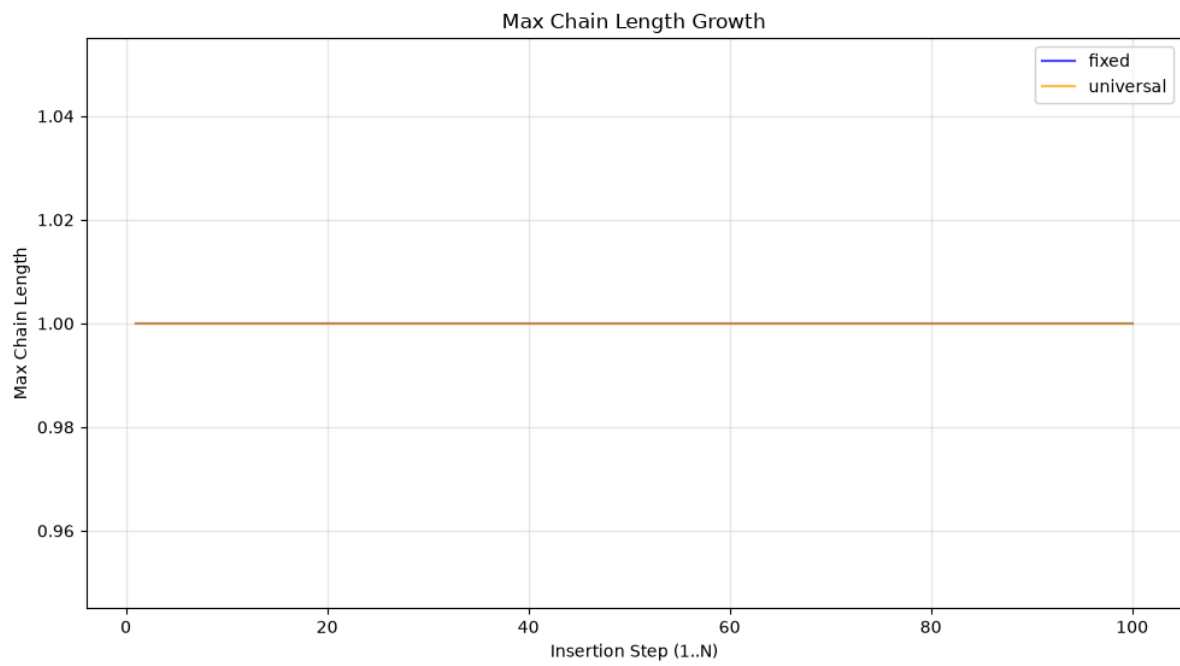
Because both hash functions successfully distribute the 100 unique keys across a table equipped with 100,000 buckets, the load factor is essentially 0.001. At this extreme sparsity, collisions are non-existent. The max chain length never exceeds 1, and total collisions remain exactly at zero for both strategies, creating perfectly overlapping flat lines.

Evolution of Max Chain Length (N=10 to N=100,000):

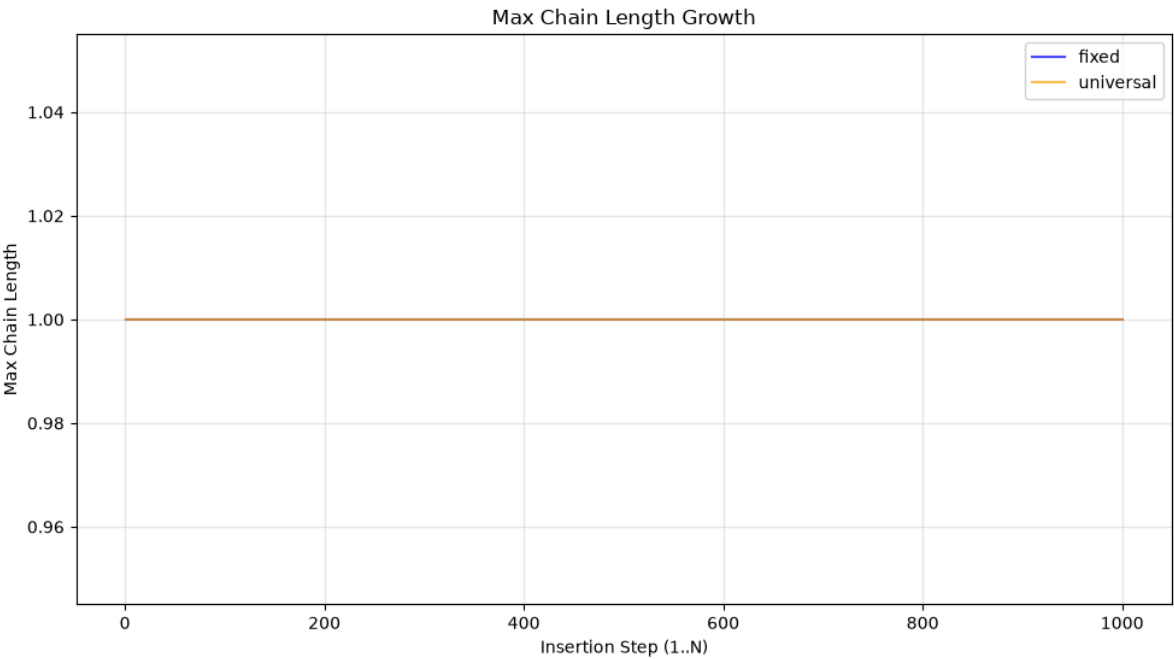
Dataset: Near_Duplicate | n = 10 (Max Chain Growth, LF=1.0)



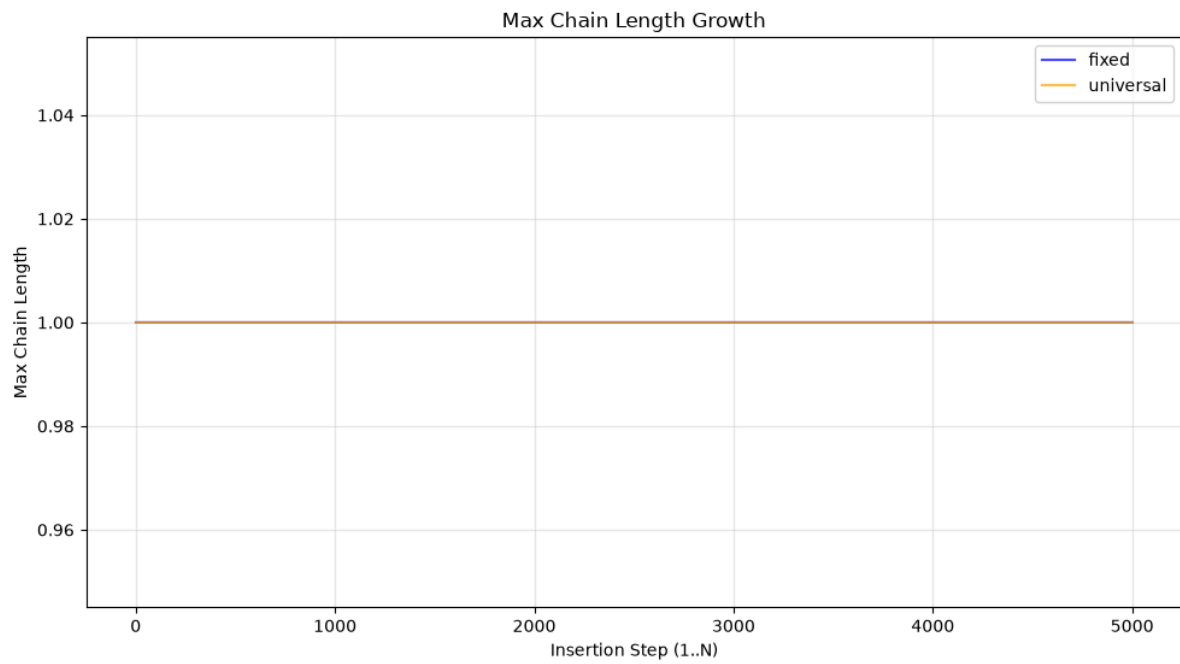
Dataset: Near_Duplicate | n = 100 (Max Chain Growth, LF=1.0)



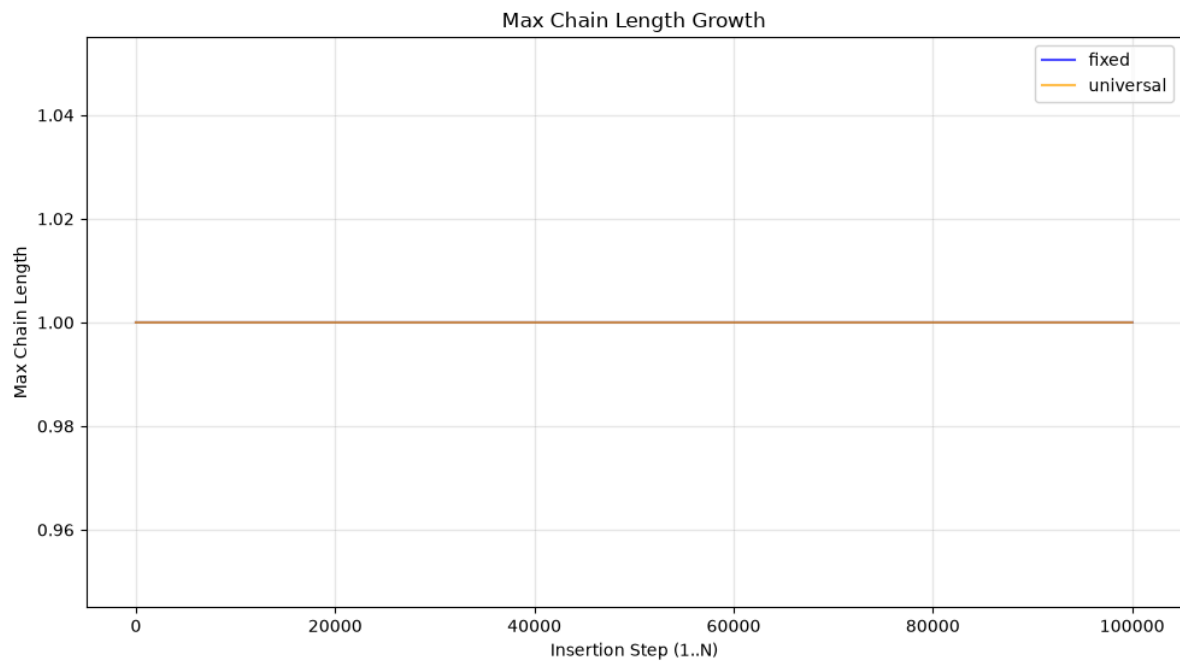
Dataset: Near_Duplicate | n = 1000 (Max Chain Growth, LF=1.0)



Dataset: Near_Duplicate | n = 5000 (Max Chain Growth, LF=1.0)

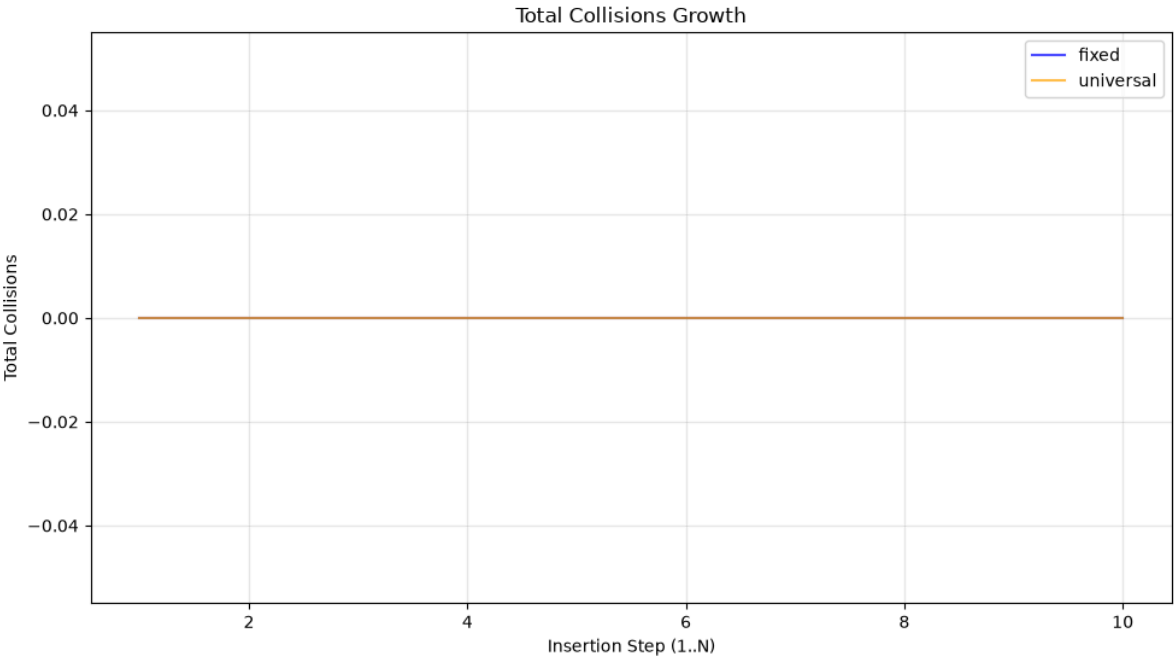


Dataset: Near_Duplicate | n = 100000 (Max Chain Growth, LF=1.0)

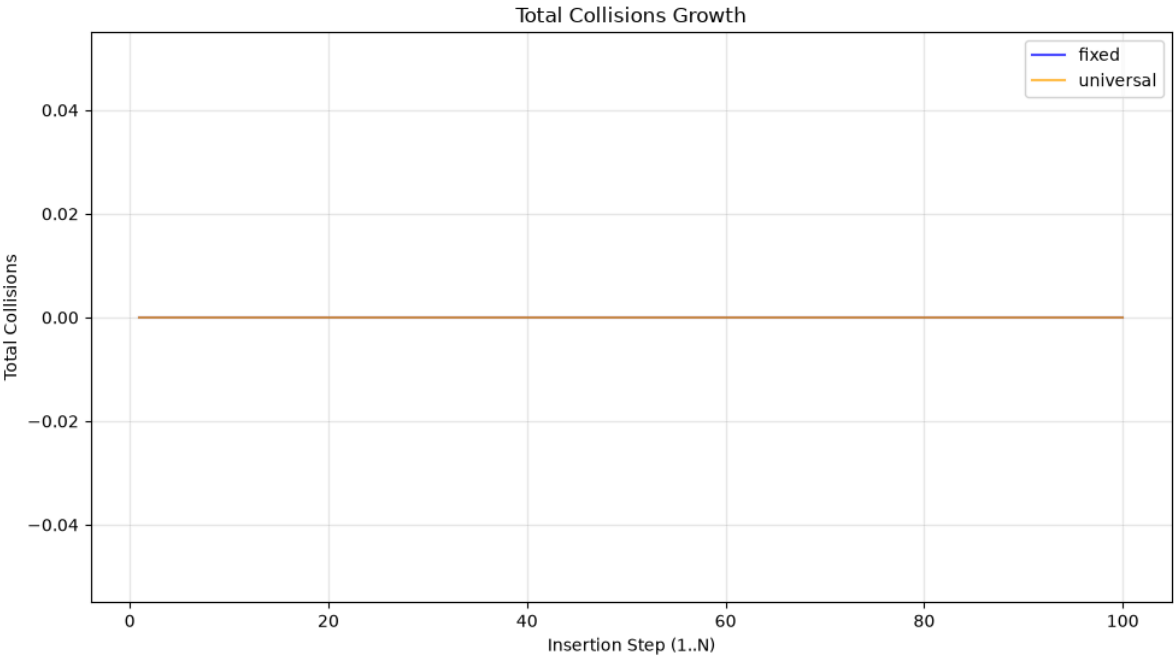


Evolution of Total Collisions (N=10 to N=100,000):

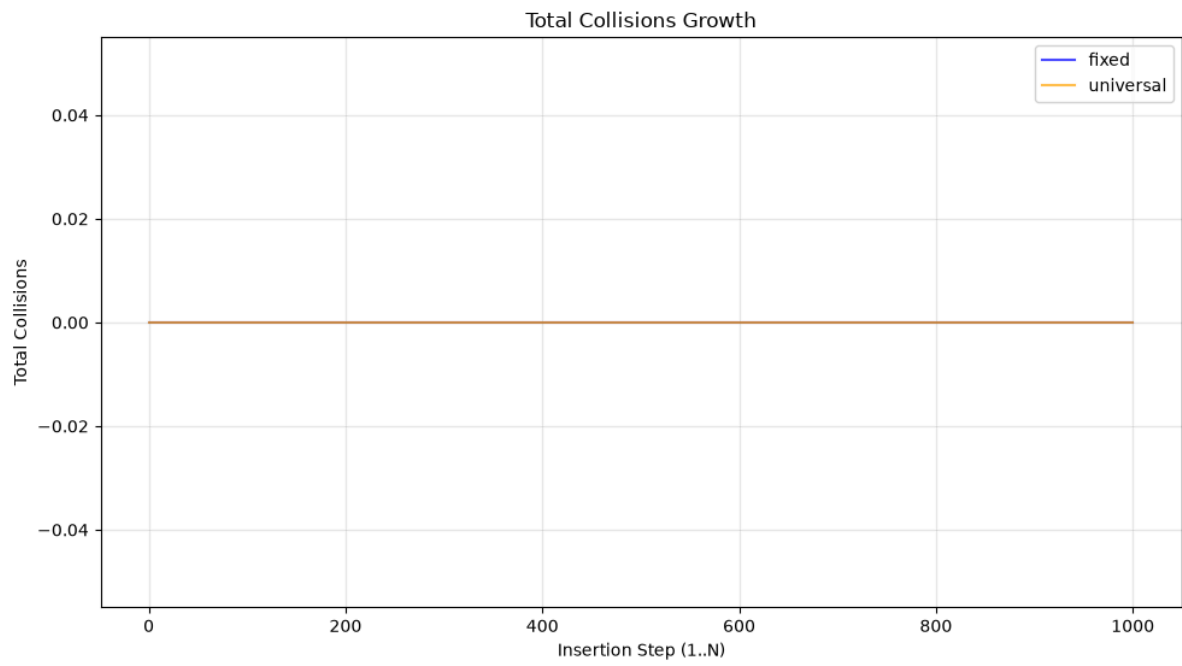
Dataset: Near_Duplicate | n = 10 (Total Collisions, LF=1.0)

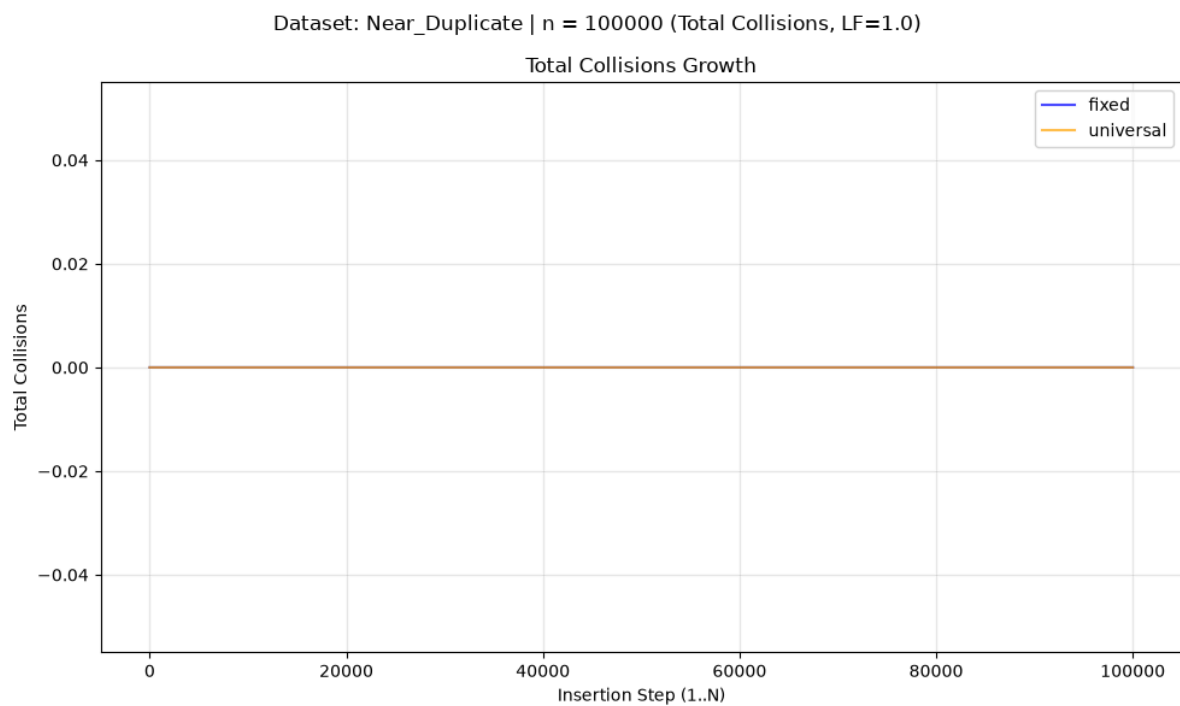
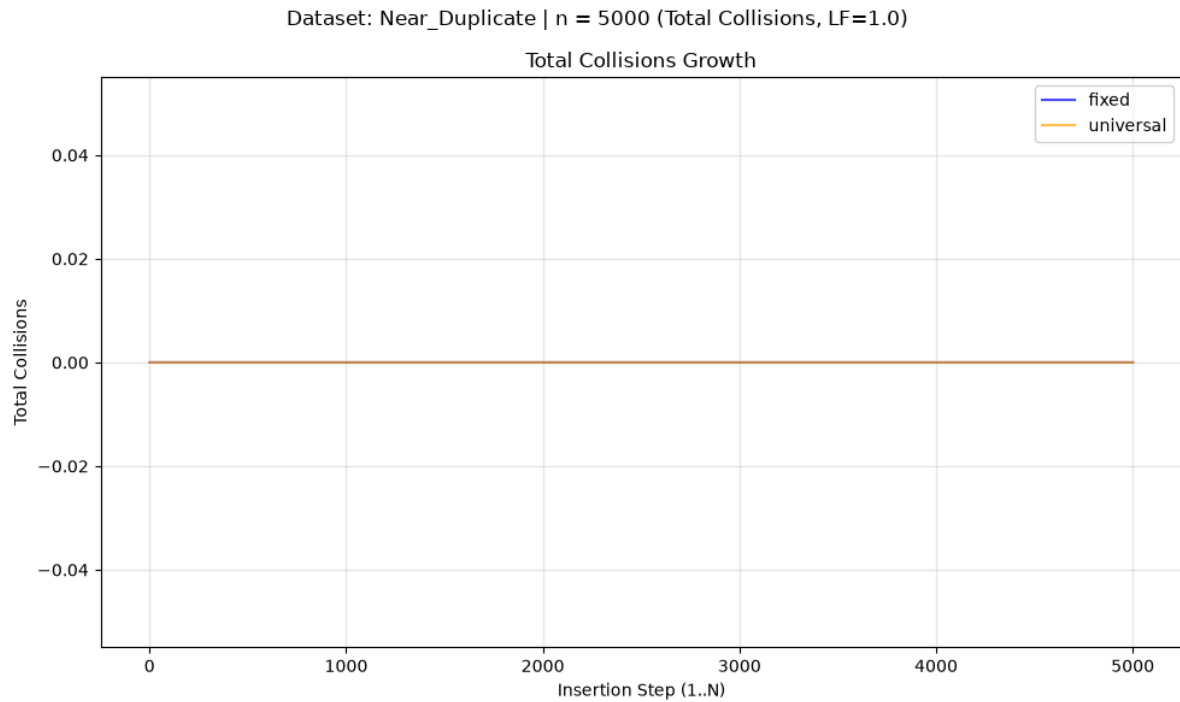


Dataset: Near_Duplicate | n = 100 (Total Collisions, LF=1.0)



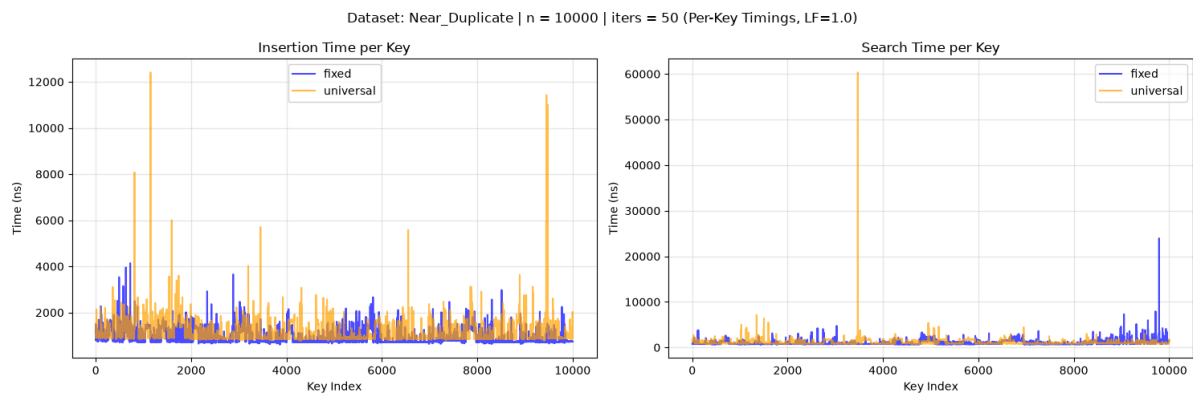
Dataset: Near_Duplicate | n = 1000 (Total Collisions, LF=1.0)





8.3 Detailed Timings Analysis

With almost empty buckets, the traversal times are instantaneous. Both insertion and search operations exhibit pristine $O(1)$ performance profiles.



Deduction: When data contains massive repetition, hash tables operate at peak efficiency due to automatic deduplication. In sparse environments, the choice of hash function becomes irrelevant as structural collisions are avoided purely by the low population density.

9. Comparative Analysis and Synthesis

9.1 Performance vs Security Trade-offs

The data clearly delineates a critical trade off in modern computer science. A simple Fixed Hash function is incredibly fast and exceptionally well suited for dense, sequential, or highly uniform data. However, it operates on an assumption of trust. The moment that trust is broken, as demonstrated in the Adversarial case, the system fails catastrophically.

9.2 The Cost of Randomness

Universal Hashing introduces a degree of randomness that acts as an insurance policy. While it occasionally causes minor, natural collisions in perfectly sequential datasets (where the Fixed Hash would have achieved a perfect distribution), this structural cost is negligible compared to the protection it offers. The $O(\log N)$ chain lengths produced by Universal Hashing are entirely acceptable and do not significantly impact the overarching $O(1)$ time complexity. The mathematical overhead of calculating $((ak + b) \bmod p) \bmod m$ is higher than $k \bmod m$, but through the use of Mersenne primes, this overhead is minimized to a few nanoseconds.

10. Conclusion

This exhaustive, large scale empirical analysis unequivocally validates the necessity of Universal Hash Families in robust software engineering. Through granular tracking of collision trajectories, max chain length evolution, and per key nanosecond timings, we have mapped the exact structural breakdown of deterministic hash tables.

The results prove that while deterministic hashing is optimal for closed, benign systems, it is critically unsuitable for public facing infrastructure. The Adversarial tests demonstrated a total collapse from $O(1)$ to $O(N^2)$ execution times, providing a clear visualization of an Algorithmic Denial of Service attack.

Conversely, Universal Hashing proved to be universally resilient. By injecting runtime randomness, it successfully dismantled the adversarial structure, mathematically neutralizing the threat and guaranteeing stable, flat, $O(1)$ performance across all tested scales and distributions. It is the definitive industry standard for secure data structure implementation.

11. References

- Carter, J. L., & Wegman, M. N. (1979). Universal classes of hash functions. Journal of computer and system sciences.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to algorithms (3rd ed.). MIT press.
- Crosby, S. A., & Wallach, D. S. (2003). Denial of Service via Algorithmic Complexity Attacks. USENIX Security Symposium.
- Knuth, D. E. (1998). The Art of Computer Programming, Volume 3: Sorting and Searching. Addison Wesley.
- Aumasson, J. P., & Bernstein, D. J. (2012). SipHash: a fast short input PRF. Progress in Cryptology – INDOCRYPT.